

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN - TIN

ĐỒ ÁN TỐT NGHIỆP

Xây dựng Chatbot hỏi đáp quy chế sinh viên

LÊ QUANG ĐỨC

Email: duclq227221@sis.hust.edu.vn

Mã sinh viên: 20227221

Chuyên ngành Hệ thống thông tin quản lý

Giảng viên hướng dẫn : TS. Ngô Thị Hiền

Chữ ký GVHD

HÀ NỘI, 6/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN - TIN

ĐỒ ÁN TỐT NGHIỆP

Xây dựng Chatbot hỏi đáp quy chế sinh viên

LÊ QUANG ĐỨC

Email: duclq227221@sis.hust.edu.vn

Mã sinh viên: 20227221

Chuyên ngành Hệ thống thông tin quản lý

Giảng viên hướng dẫn : TS. Ngô Thị Hiền

Chữ ký GVHD

HÀ NỘI, 6/2026

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

1. Mục tiêu và nội dung của đồ án

(a) Mục tiêu: Nghiên cứu đề xuất triển khai hệ thống chatbot theo kiến trúc Agentic RAG.

(b) Nội dung: Nghiên cứu lý thuyết nền tảng LLM, RAG, Agent - Thiết kế, cài đặt đồ thị Lang Graph gồm các node.

2. Kết quả đạt được:

- Hệ thống chatbot với kiến trúc ba tầng
- Công trên xử lý dữ liệu và công hậu xử lý giúp hiển số liệu vào vẽ đầu ra

3. Ý thức làm việc của sinh viên:

- Ý thức làm việc tốt

Hà Nội, tháng 6 năm 2026

Giảng viên hướng dẫn



TS. Ngô Thị Hiền

PHIẾU BÁO CÁO TIẾN ĐỘ ĐỒ ÁN



ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN - TIN

[Lớp học](#) [Điểm thi](#) [Bảo lỗi](#) [Phúc khảo](#) [Lịch thi](#) [Thi LT](#) [Quy định/Thông báo](#)

D

Danh sách lớp học / Đồ án tốt nghiệp cử nhân / 761517

Thông tin lớp học mã 761517

Kì học: 20252

Mã học phần: MI4900

Tên học phần: Đồ án tốt nghiệp cử nhân

Mã lớp: 761517

Đồ án

Giáo viên hướng dẫn:

Ngô Thị Hiền

Tên đồ án:

Xây dựng Chatbot hỏi đáp quy chế sinh viên

Nội dung:

Xây dựng Chatbot hỏi đáp quy chế sinh viên ứng dụng thực tế với độ chính xác cao

Các mốc kiểm soát chính:

Ver 01: 04/2026

Ver 02: 05/2026

ver final: 06/2026

Giáo viên phân biên:

Danh sách đánh giá đồ án

Ngày đánh giá	Lần	Nội dung kế hoạch	Nội dung đã thực hiện	Điểm tích cực	Điểm nội dung	Ghi chú
21/04/2026	1	Thực hiện kế hoạch hoàn thành ver 01 về đề tài theo yêu cầu đặt ra	Hoàn thành ver 01	9.5	9.5	
30/05/2026	2	Thực hiện kế hoạch hoàn thành ver 02 về đề tài theo yêu cầu đặt ra	Hoàn thành ver 02	9.5	9.5	

Lời cảm ơn

Trước tiên, em xin gửi lời cảm ơn chân thành đến TS. Ngô Thị Hiền đã tận tình hướng dẫn, định hướng và đưa ra những góp ý quý báu trong suốt quá trình thực hiện đồ án. Những phản biện và gợi mở của cô đã giúp em nhìn nhận vấn đề sâu sắc hơn và hoàn thiện hệ thống một cách có hệ thống.

Em cũng xin gửi lời tri ân sâu sắc đến các thầy cô của Khoa Toán - Tin, Đại học Bách khoa Hà Nội, những người đã trang bị cho em nền tảng kiến thức vững chắc để em có thể hoàn thành đồ án này.

Trong quá trình thực hiện đồ án và xử lý văn bản, chắc chắn em khó tránh khỏi những hạn chế và thiếu sót. Em rất mong nhận được sự góp ý của các thầy cô cũng như các độc giả để đồ án được hoàn thiện hơn.

Tóm tắt đồ án

- **Mục tiêu và vấn đề giải quyết:** Xây dựng hệ thống Chatbot ứng dụng kiến trúc Agentic RAG để hỗ trợ sinh viên tra cứu quy chế đào tạo Đại học Bách khoa Hà Nội. Đồ án giúp giải quyết khó khăn khi tra cứu thông tin từ các file PDF phân tán và khắc phục điểm yếu của RAG truyền thống trong việc xử lý câu hỏi phức tạp, thiếu ngữ cảnh.
- **Phương pháp và công nghệ triển khai:** Hệ thống vận hành dựa trên đồ thị trạng thái LangGraph. Kiến trúc tích hợp bốn công cụ cốt lõi (tìm kiếm, đánh giá, viết lại, tổng hợp) và được bảo vệ bởi hai cổng kiểm soát chặt chẽ: Intent Gate - phân loại ý định và Confidence Gate - chống ảo giác.
- **Kết quả và hướng phát triển:** Chatbot xử lý thành công đa dạng các kịch bản truy vấn. Đồ án giúp sinh viên làm chủ quy trình phát triển AI Agent end-to-end và mở ra hướng nâng cấp tiềm năng như: tích hợp tìm kiếm lai, triển khai bằng Docker và tối ưu cơ sở dữ liệu có cấu trúc.

Hà Nội, tháng 6 năm 2026
Tác giả đồ án

Lê Quang Đức

Danh sách hình vẽ

1.1	Kiến trúc mô hình Transformer [7]	12
1.2	Rule-based chatbot	16
1.3	AI-based chatbot	16
1.4	Kiến trúc RAG [3]	17
1.5	Kiến trúc Agentic RAG [6]	19
2.1	Hệ thống chatbot của trường	24
2.2	Hệ thống chưa có khả năng truy vết	25
2.3	Sơ đồ kiến trúc tổng thể	28
2.4	Tài liệu có nhiều thông tin nhiễu	29
2.5	Dữ liệu có cấu trúc chặt chẽ	30
2.6	Dữ liệu dạng bảng	30
2.7	Cấu trúc phân tầng dữ liệu	30
2.8	Intent Schema	32
2.9	Luồng phân loại Intent ba tầng	33
2.10	Tầng 1: LLM Extraction	33
2.11	Đồ thị trạng thái	38
2.12	Câu truy vấn dùng LLM đánh giá	40
2.13	Câu truy vấn dùng LLM viết lại câu	40
2.14	LLM tổng hợp và tạo câu trả lời hoàn chỉnh	41
2.15	Lớp hội thoại	42
3.1	Cấu hình hệ thống	44
3.2	Tham số mô hình ngôn ngữ lớn	45
3.3	Giao diện hệ thống	45
3.4	UC1: Hỏi về chính sách học bổng	46
3.5	Hỏi đáp nối tiếp dựa trên ngữ cảnh	47
3.6	Hỏi về học phí sẽ cần thông tin về ngành học	47
3.7	Hệ thống từ chối trả lời do không có thông tin	48
3.8	Hệ thống từ chối trả lời do không có thông tin	48
3.9	Kết quả phân tích	49
3.10	Kết quả phân tích	49
3.11	Kết quả phân tích	49
3.12	Hệ thống từ chối trả lời	50
3.13	Khả năng truy vết	50

Danh sách bảng

- 1.1 So sánh đặc điểm giữa RAG Truyền thống và Agentic RAG 20
- 3.1 Cấu hình phần cứng 43
- 3.2 Tùy chọn sử dụng theo hai chế độ 45

Mục lục

MỞ ĐẦU	10
1 Cơ sở lý thuyết	11
1.1 Tổng quan về mô hình ngôn ngữ lớn	11
1.1.1 Kiến trúc Transformer và nguyên lý hoạt động	11
1.1.2 Ưu điểm và thách thức	14
1.2 Tổng quan về Chatbot	15
1.2.1 Khái niệm	15
1.2.2 Phân loại Chatbot	16
1.3 Retrieval-Augmented Generation	17
1.3.1 Khái niệm	17
1.3.2 Hạn chế	18
1.4 Agentic RAG	19
1.4.1 Khái niệm	19
1.4.2 Sự khác biệt giữa RAG và Agentic RAG	20
1.5 Tech stack và các kỹ thuật sử dụng	20
1.5.1 Tech stack	20
1.5.2 Các kỹ thuật sử dụng	21
2 Phân tích và thiết kế hệ thống	23
2.1 Khảo sát thực tế	23
2.1.1 Hệ thống chatbot hiện tại	23
2.1.2 Ưu nhược điểm của hệ thống hiện tại	24
2.2 Phân tích bài toán và yêu cầu hệ thống	25
2.2.1 Phân tích bài toán	25
2.2.2 Yêu cầu chức năng	26
2.2.3 Yêu cầu phi chức năng	27
2.3 Kiến trúc tổng thể hệ thống	27
2.3.1 Sơ đồ kiến trúc tổng thể	27
2.4 Thiết kế cơ sở dữ liệu	29
2.4.1 Khảo sát và đặc tả dữ liệu nguồn	29
2.4.2 Xây dựng cơ sở tri thức	30
2.5 Thiết kế tầng xử lý dữ liệu	31
2.5.1 Cổng tiền xử lý	32
2.5.2 Cổng hậu xử lý	34
2.6 Thiết kế tầng suy luận tác tử	35
2.6.1 Điều phối bằng đồ thị trạng thái	35
2.6.2 Thiết kế các công cụ hỗ trợ	38
2.7 Thiết kế tầng bộ nhớ	41

2.7.1	Cơ sở dữ liệu vector — Tri thức tĩnh	41
2.7.2	Bộ nhớ hội thoại — Tri thức động	41
3	Thực nghiệm và đánh giá	43
3.1	Môi trường và cấu hình triển khai	43
3.1.1	Cấu hình phần cứng và phần mềm	43
3.1.2	Cấu hình mô hình	44
3.2	Các kịch bản thử nghiệm	46
3.2.1	Truy vấn đơn giản	46
3.2.2	Hội thoại đa lượt	47
3.2.3	Câu hỏi ngoài phạm vi	48
3.3	Phương pháp đánh giá	49
3.3.1	Đánh giá định lượng	49
3.3.2	Đánh giá định tính	50
	Kết luận	51
	Tài liệu tham khảo	53

Lời mở đầu

Trong bối cảnh chuyển đổi số giáo dục, việc tiếp cận thông tin học vụ chính xác là nhu cầu cấp thiết. Đặc biệt tại Đại học Bách Khoa Hà Nội, với đặc thù quy mô đào tạo lớn và hệ thống quy chế tín chỉ chặt chẽ, sinh viên thường xuyên đối mặt với thách thức trong việc tra cứu các văn bản hành chính. Khối lượng tài liệu đồ sộ, sự phân tán của các nguồn dữ liệu và ngôn ngữ văn bản phức tạp khiến việc tìm kiếm thông tin trở nên tốn thời gian, dễ dẫn đến những hiểu lầm ảnh hưởng trực tiếp đến lộ trình học tập của sinh viên Bách Khoa.

Sự bùng nổ của các mô hình ngôn ngữ lớn (LLM) mang lại tiềm năng to lớn cho các hệ thống hỏi đáp tự động. Tuy nhiên, các mô hình phổ quát hiện nay chưa thể đáp ứng tốt nhu cầu đặc thù của môi trường đại học. Chúng thường gặp vấn đề "ảo giác" - tự sinh thông tin sai lệch do thiếu dữ liệu nội bộ, và thiếu đi sự thấu hiểu về ngữ cảnh cũng như văn phong giao tiếp gần gũi cần thiết cho sinh viên.

Xuất phát từ thực tiễn đó, đề tài "**Xây dựng chatbot hỏi đáp quy chế sinh viên**" được thực hiện nhằm nâng cấp toàn diện hệ thống chatbot quy chế theo kiến trúc Agentic RAG - nơi một tác tử thông minh được điều phối bởi đồ thị trạng thái có khả năng tự suy luận, tự đánh giá, và tự sửa sai qua nhiều vòng lặp. Cụ thể, hệ thống được thiết kế với các cơ chế chủ chốt: Cổng tiền xử lý phân loại ý định và thu thập thông tin còn thiếu từ người dùng; vòng lặp đa bước cho phép tác tử viết lại truy vấn và tìm kiếm lại khi tài liệu chưa đủ chất lượng; Cổng hậu xử lý tính toán điểm tin cậy và chặn đứng các câu trả lời không đạt ngưỡng; cùng với bộ nhớ thoại giúp hệ thống duy trì ngữ cảnh xuyên suốt cuộc hội thoại đa lượt. Toàn bộ cơ sở tri thức được xây dựng trên dữ liệu quy chế chính thức của Nhà trường, đảm bảo mọi câu trả lời đều có trích dẫn nguồn cụ thể và có thể kiểm chứng.

Ngoài phần mở đầu, kết luận và tài liệu tham khảo, nội dung chính của đồ án được trình bày trong ba chương sau:

- **Chương 1: Cơ sở lý thuyết:** Trình bày các khái niệm nền tảng về Mô hình Ngôn ngữ Lớn, Chatbot, kiến trúc RAG và bước tiến hóa lên Agentic RAG, mô hình nhúng, cơ sở dữ liệu vector, cùng framework điều phối đồ thị LangGraph.
- **Chương 2: Phân tích và thiết kế hệ thống:** Đi sâu vào phân tích bài toán, thiết kế kiến trúc ba tầng, quy trình xây dựng cơ sở tri thức từ tài liệu PDF, thiết kế đồ thị trạng thái và các công cụ hỗ trợ, cơ chế tính điểm tin cậy, và tầng bộ nhớ hội thoại.
- **Chương 3: Thực nghiệm và đánh giá:** Trình bày môi trường triển khai, các kịch bản kiểm thử trên bộ Use Case thực tế (truy vấn đơn giản, truy vấn phức tạp, hội thoại đa lượt, câu hỏi ngoài phạm vi), phân tích kết quả và đánh giá hiệu năng hệ thống.

Chương 1

Cơ sở lý thuyết

1.1 Tổng quan về mô hình ngôn ngữ lớn

Trong những năm gần đây, lĩnh vực xử lý ngôn ngữ tự nhiên (NLP) đã chứng kiến một bước chuyển mình mang tính cách mạng với sự ra đời của các mô hình ngôn ngữ lớn (LLMs). Đây là các mô hình học sâu (Deep Learning) được huấn luyện trên một lượng dữ liệu văn bản khổng lồ, sở hữu hàng tỷ, thậm chí hàng nghìn tỷ tham số. Thay vì được huấn luyện cho một tác vụ cụ thể, LLMs được huấn luyện để dự đoán từ tiếp theo trong một chuỗi, một nhiệm vụ tưởng chừng đơn giản nhưng lại giúp mô hình học được các cấu trúc ngữ pháp, ngữ nghĩa, và một lượng lớn kiến thức về thế giới.

Sự phát triển của LLMs đã mở ra những khả năng vượt bậc trong việc hiểu và tạo sinh ngôn ngữ tự nhiên, cho phép máy tính thực hiện các tác vụ phức tạp như trả lời câu hỏi, tóm tắt văn bản, dịch thuật, và thậm chí là lập trình, với độ chính xác và mạch lạc đáng kinh ngạc. Nền tảng kiến trúc cốt lõi đằng sau sự thành công của hầu hết các LLM hiện đại chính là kiến trúc Transformer.

1.1.1 Kiến trúc Transformer và nguyên lý hoạt động

Trước năm 2017, các mô hình NLP hàng đầu chủ yếu dựa vào các kiến trúc tuần tự như mạng nơ-ron hồi quy (RNN) hay mạng bộ nhớ dài-ngắn hạn (LSTM). Mặc dù hiệu quả, các mô hình này gặp hai hạn chế lớn:

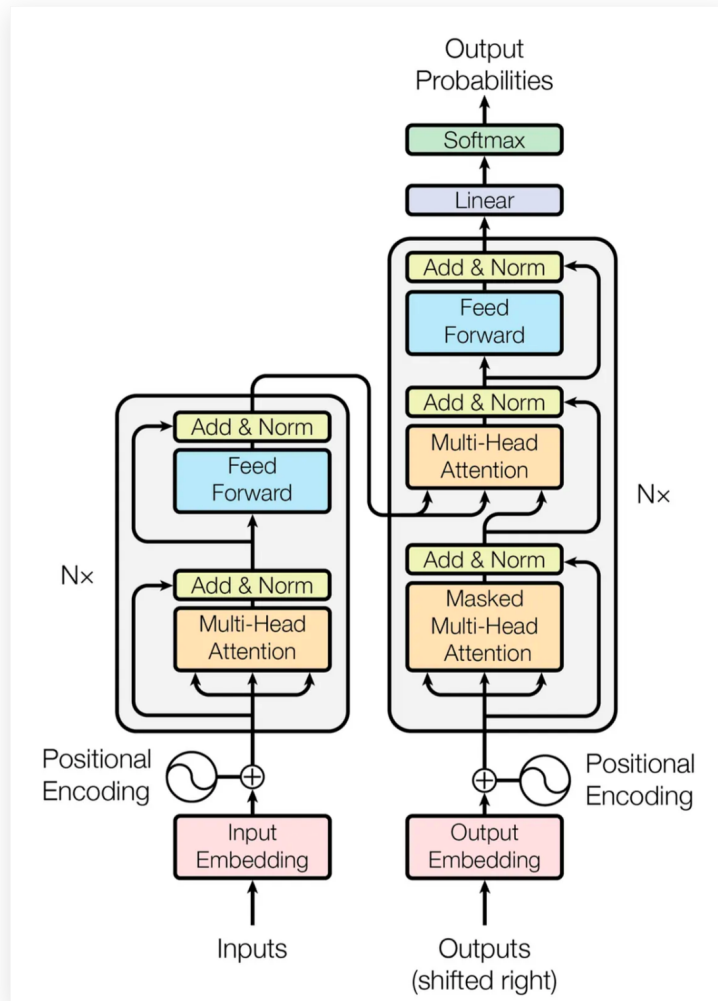
1. Khó khăn trong việc nắm bắt các phụ thuộc ngữ cảnh ở xa (long-range dependencies) do phải xử lý văn bản theo từng từ.
2. Khó khăn trong việc song song hóa quá trình huấn luyện.

Sự ra đời của kiến trúc Transformer trong bài báo "Attention Is All You Need" (Vaswani et al., 2017) đã giải quyết triệt để cả hai vấn đề này. Transformer loại bỏ hoàn toàn cơ chế hồi quy và chỉ dựa vào một cơ chế duy nhất gọi là **Self-Attention (Tự chú ý)**.

Transformer tuân theo kiến trúc tổng thể encoder-decoder.

1. **Encoder (Bộ mã hóa):** Nằm ở nửa bên trái của 1.1, bộ mã hóa có nhiệm vụ ánh xạ một chuỗi đầu vào gồm các biểu diễn ký hiệu $(x_1, \dots, x_n)$ thành một chuỗi các biểu diễn liên tục $z = (z_1, \dots, z_n)$.
2. **Decoder (Bộ giải mã):** Nằm ở nửa bên phải của 1.1, bộ giải mã nhận đầu ra z từ encoder và tạo ra một chuỗi đầu ra $(y_1, \dots, y_m)$, từng phần tử một. Mô hình này có tính

chất tự hồi quy (auto-regressive), nghĩa là nó tiêu thụ các ký hiệu đã tạo ra trước đó làm đầu vào bổ sung khi tạo ra ký hiệu tiếp theo.



Hình 1.1: Kiến trúc mô hình Transformer [7]

Cả bộ mã hóa và giải mã đều được cấu tạo từ các khối (stacks) gồm $N = 6$ tầng (layers) giống hệt nhau.

Tầng Encoder

Mỗi tầng trong bộ mã hóa có hai tầng phụ (sub-layers):

1. **Multi-Head Self-Attention:** Đây là nơi chuỗi đầu vào "tự chú ý" đến chính nó. Nó giúp mô hình xác định xem mỗi từ trong câu nên "chú ý" đến những từ nào khác trong cùng câu đó để hiểu rõ ngữ cảnh.
2. **Position-wise Feed-Forward Network:** Một mạng nơ-ron truyền thẳng (FFN) 2 lớp đơn giản.

Một điểm cực kỳ quan trọng là sau mỗi lớp con, mô hình đều áp dụng một kết nối phần dư (residual connection) và chuẩn hóa tầng (layer normalization). Tức là đầu ra sẽ là

$LayerNorm(x + Sublayer(x))$. Việc này giúp huấn luyện các mô hình mà không bị hiện tượng "vanishing gradients" (gradient triệt tiêu).

Tầng Decoder

Mỗi tầng trong bộ giải mã cũng bao gồm $N = 6$ tầng giống hệt nhau. Ngoài hai tầng phụ như trong mỗi tầng encoder, bộ giải mã chèn thêm một tầng phụ thứ ba:

1. **Masked Multi-Head Self-Attention:** Tầng này được sửa đổi để ngăn các vị trí chú ý đến các vị trí tiếp theo. Việc che giấu (masking) này đảm bảo rằng dự đoán cho vị trí i chỉ có thể phụ thuộc vào các đầu ra đã biết ở các vị trí trước i , bảo toàn tính chất tự hồi quy.
2. **Multi-Head Attention (Encoder-Decoder):** Tầng phụ thứ hai thực hiện cơ chế chú ý đa đầu (multi-head attention) trên đầu ra của khối encoder. Điều này cho phép mọi vị trí trong decoder chú ý đến tất cả các vị trí trong chuỗi đầu vào.
3. **Mạng Feed-Forward (Position-wise):** Tương tự như trong encoder.

Tương tự Encoder, mỗi lớp con này đều có kết nối phần dư và chuẩn hóa tầng.

Scaled Dot-Product Attention

Một hàm chú ý có thể được mô tả là ánh xạ một truy vấn (Q) và một tập hợp các cặp khóa-giá trị ($K - V$) tới một đầu ra. Đầu ra được tính bằng tổng có trọng số của các giá trị (V), trong đó trọng số gán cho mỗi giá trị được tính bằng hàm tương thích giữa truy vấn (Q) với khóa (K) tương ứng.

Cơ chế chú ý này được gọi là "Scaled Dot-Product Attention". Đầu vào bao gồm các truy vấn (Q) và khóa (K) có chiều d_k , và các giá trị (V) có chiều d_v . Cơ chế này tính toán tích vô hướng của truy vấn với tất cả các khóa, chia mỗi kết quả cho $\sqrt{d_k}$, và sau đó áp dụng hàm softmax để thu được trọng số cho các giá trị.

Công thức tính toán attention là:

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1.1)$$

Việc chia cho $\sqrt{d_k}$ là một yếu tố tỷ lệ (scaling factor). Khi d_k lớn, tích vô hướng có thể trở nên rất lớn, đẩy hàm softmax vào vùng có gradient cực nhỏ. Việc chia tỷ lệ này giúp chống lại hiệu ứng đó.

Multi-Head Attention

Thay vì thực hiện một hàm attention duy nhất với các khóa, giá trị và truy vấn có kích thước d_{model} , Transformer thấy lợi ích khi chiếu tuyến tính các truy vấn, khóa và giá trị h lần bằng các phép chiếu tuyến tính được học khác nhau.

1. **Phép Chiếu:** Các truy vấn, khóa và giá trị được chiếu tuyến tính h lần tới các chiều d_k, d_k, d_v tương ứng.

⁰Nội dung tham khảo từ bài báo "Attention Is All You Need" (Vaswani et al., 2017)[7].

2. **Thực hiện song song:** Hàm attention được thực hiện song song trên mỗi phiên bản được chiếu này, tạo ra h đầu ra d_v -chiều (gọi là "heads").
3. **Kết hợp:** Các đầu ra này được nối lại và sau đó được chiếu tuyến tính một lần nữa để tạo ra giá trị cuối cùng.

Multi-head attention cho phép mô hình cùng lúc chú ý đến thông tin từ các không gian biểu diễn khác nhau tại các vị trí khác nhau. Nếu chỉ dùng một đầu chú ý việc lấy trung bình sẽ cản trở khả năng này.

Transformer sử dụng multi-head attention theo ba cách:

1. **Encoder-Decoder Attention:** Truy vấn (Q) đến từ tầng decoder trước đó, còn khóa (K) và giá trị (V) đến từ đầu ra của encoder.
2. **Encoder Self-Attention:** Tất cả Q, K, V đều đến từ đầu ra của tầng encoder trước đó.
3. **Decoder Self-Attention (Masked):** Tất cả Q, K, V đều đến từ đầu ra của tầng decoder trước đó, nhưng được che giấu (masking) để ngăn dòng thông tin "nhìn về tương lai" (leftward information flow).

Position-wise Feed-Forward Networks

Ngoài các tầng phụ chú ý, mỗi tầng trong encoder và decoder đều chứa một mạng nơ-ron feed-forward kết nối đầy đủ. Mạng này được áp dụng cho từng vị trí một cách riêng biệt và giống hệt nhau. Nó bao gồm hai phép biến đổi tuyến tính với một hàm kích hoạt ReLU ở giữa.

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (1.2)$$

Mặc dù các phép biến đổi tuyến tính là giống nhau trên các vị trí khác nhau, chúng sử dụng các tham số khác nhau giữa các tầng.

1.1.2 Ưu điểm và thách thức

Các mô hình ngôn ngữ lớn, được xây dựng trên kiến trúc Transformer, đã mang lại một cuộc cách mạng trong lĩnh vực trí tuệ nhân tạo. Tuy nhiên, bên cạnh những lợi ích vượt trội, việc ứng dụng chúng cũng đi kèm với nhiều thách thức đáng kể.

Ưu điểm mô hình ngôn ngữ lớn:

LLMs sở hữu nhiều khả năng đột phá, khiến chúng trở nên hữu ích trong hàng loạt ứng dụng:

- **Tính đa dụng và linh hoạt:** Một trong những ưu điểm lớn nhất là khả năng thực hiện nhiều tác vụ khác nhau chỉ với một mô hình duy nhất. Các tác vụ này bao gồm dịch thuật, tóm tắt văn bản, trả lời câu hỏi, sáng tạo nội dung, và thậm chí là tạo sinh mã nguồn lập trình.
- **Tự động hóa và nâng cao hiệu suất:** Tự động hóa các tác vụ lặp đi lặp lại và tốn thời gian. Điều này giúp giải phóng sức lao động của con người cho các công việc mang tính chiến lược hơn.

- **Khả năng học trong ngữ cảnh (In-context Learning):** LLMs thể hiện khả năng học hỏi nhanh chóng, đặc biệt là học theo ngữ cảnh (hay còn gọi là zero-shot và few-shot learning). Chúng có thể thực hiện các tác vụ mới mà không cần huấn luyện lại hay tinh chỉnh chuyên biệt, mà chỉ cần một vài ví dụ hoặc mô tả làm đầu vào.

Thách thức và hạn chế

Mặc dù có nhiều ưu điểm, việc triển khai và ứng dụng LLMs phải đối mặt với nhiều rào cản lớn:

- **Hiện tượng "Ảo giác" (Hallucination):** LLMs có xu hướng tạo ra các thông tin nghe có vẻ hợp lý nhưng lại hoàn toàn sai sự thật, không dựa trên dữ liệu đầu vào hoặc không tương thích với ý định của người dùng. Điều này làm giảm đáng kể độ tin cậy của mô hình, đặc biệt trong các lĩnh vực yêu cầu tính chính xác cao như y tế hay pháp lý.
- **Chi phí tính toán và tài nguyên:** Việc huấn luyện các mô hình với hàng trăm tỷ tham số đòi hỏi nguồn tài nguyên tính toán khổng lồ, đặt ra các vấn đề về chi phí và tính bền vững của công nghệ. Quá trình triển khai và thực thi (inference) cũng rất phức tạp và tốn kém.
- **Rủi ro về an toàn và bảo mật:**
 - **Quyền riêng tư:** LLMs có thể vô tình làm rò rỉ thông tin cá nhân, nội bộ có trong dữ liệu huấn luyện hoặc trong câu lệnh của người dùng.
 - **Tấn công:** Các mô hình dễ bị tấn công bằng "prompt injection", nơi kẻ tấn công sử dụng các câu lệnh đặc biệt để lừa mô hình bỏ qua các quy tắc an toàn và tạo ra nội dung độc hại.
 - **Bản quyền:** Việc huấn luyện trên dữ liệu có bản quyền mà không có sự đồng ý đặt ra các thách thức pháp lý lớn.

1.2 Tổng quan về Chatbot

1.2.1 Khái niệm

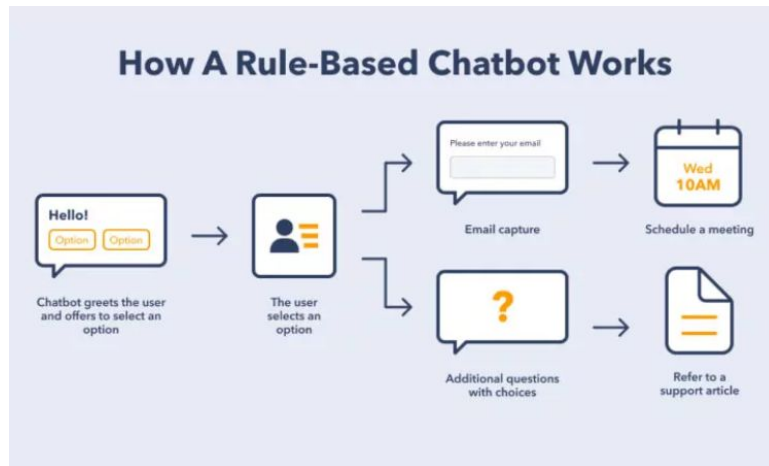
Chatbot là một chương trình phần mềm được thiết kế để mô phỏng và thực hiện các cuộc trò chuyện với người dùng bằng ngôn ngữ tự nhiên, thông qua văn bản hoặc giọng nói. Chúng đóng vai trò như các trợ lý ảo, được tích hợp vào các nền tảng như website, ứng dụng nhắn tin, hoặc các nền tảng mạng xã hội để tương tác với con người.

Lịch sử của chatbot bắt đầu từ rất sớm, với một trong những đại diện sơ khai nhất là ELIZA (1966). ELIZA hoạt động dựa trên việc nhận diện các từ khóa trong câu của người dùng để đưa ra các câu trả lời mẫu được lập trình sẵn. Mặc dù đơn giản, nó đã đặt nền móng cho ý tưởng về việc máy tính có thể giao tiếp với con người. Trải qua nhiều thập kỷ phát triển với các dấu mốc như A.L.I.C.E (1995), và sự bùng nổ của các trợ lý ảo như Siri (2010) hay Alexa, công nghệ chatbot đã có những bước tiến vượt bậc.

Để hiểu rõ hơn về cách chatbot hoạt động, chúng ta có thể phân loại chúng thành các nhóm chính dựa trên công nghệ cốt lõi

1.2.2 Phân loại Chatbot

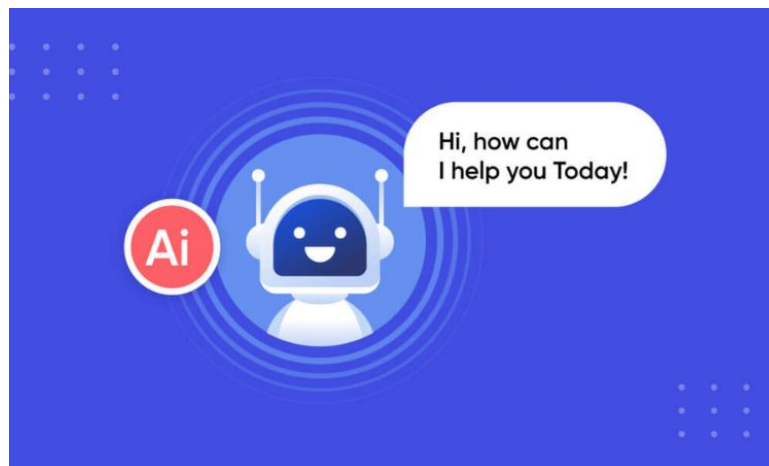
Chatbot dựa trên luật (Rule-based / Scripted Chatbots)



Hình 1.2: Rule-based chatbot

Đây là loại chatbot cơ bản và phổ biến nhất trong giai đoạn đầu. Chúng hoạt động theo một kịch bản hoặc một tập hợp các quy tắc được lập trình sẵn, thường có dạng cây quyết định. Người dùng thường tương tác bằng cách nhấp vào các nút tùy chọn hoặc chatbot sẽ nhận diện các từ khóa cụ thể. Đây là một dạng chatbot kinh điển dành cho doanh nghiệp khi họ hiểu rõ những câu hỏi của khách hàng.

Chatbot dựa trên Trí tuệ nhân tạo (AI-based Chatbots):



Hình 1.3: AI-based chatbot

Đây là thế hệ chatbot thông minh hơn, sử dụng công nghệ xử lý ngôn ngữ tự nhiên và học máy. Thay vì các quy tắc cứng, chúng được huấn luyện trên một lượng lớn dữ liệu hội thoại để "học" cách hiểu ý định và ngữ cảnh của người dùng. Chúng có khả năng tự học và cải thiện câu trả lời theo thời gian, mặc dù linh hoạt hơn chatbot dựa trên luật, các mô hình NLP truyền thống vẫn gặp khó khăn trong việc duy trì các cuộc hội thoại dài.

Sự xuất hiện của các mô hình ngôn ngữ lớn chính là bước đột phá cách mạng hóa thế hệ chatbot AI này.

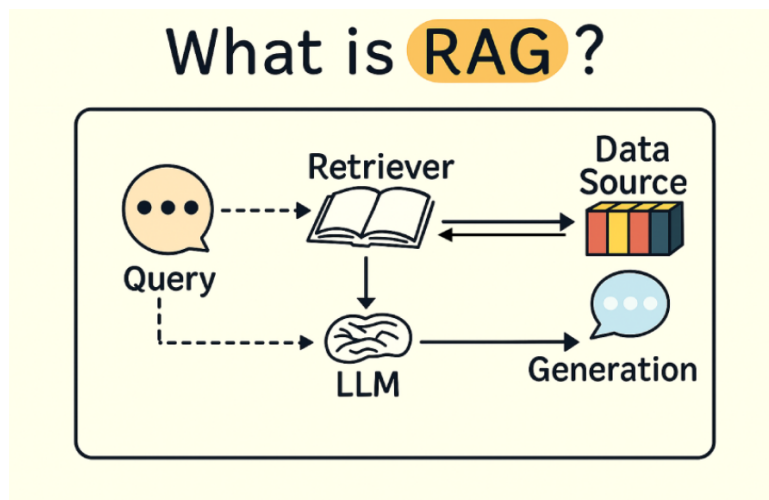
- **Chatbot ứng dụng LLM:** Thay vì chỉ truy xuất thông tin từ cơ sở dữ liệu hoặc kịch bản, các chatbot này tạo sinh ra các câu trả lời mới.
- **Tác động:** LLMs giúp chatbot hiểu được các truy vấn phức tạp, đa sắc thái và duy trì các cuộc hội thoại đa lượt một cách mạch lạc. Chúng có thể tạo ra các phản hồi linh hoạt, sáng tạo và giống với con người hơn bất kỳ công nghệ nào trước đó. Các mô hình như ChatGPT là ví dụ điển hình cho thấy khả năng vượt trội của chatbot dựa trên LLM.

Tuy nhiên, như đã phân tích, việc ứng dụng LLM trực tiếp vào chatbot cũng bộc lộ những thách thức lớn, đặc biệt là hiện tượng "ảo giác", đặc biệt là trong các lĩnh vực đòi hỏi kiến thức chuyên biệt (như y tế, pháp luật, hay tài liệu kỹ thuật của doanh nghiệp), rủi ro này là không thể chấp nhận được. Điều này dẫn đến nhu cầu phát triển các kiến trúc tiên tiến hơn, kết hợp sức mạnh tạo sinh của LLM với tính chính xác của cơ sở kiến thức bên ngoài. Đây chính là tiền đề cho kiến trúc sinh tăng cường truy xuất, sẽ được trình bày chi tiết trong phần tiếp theo.

1.3 Retrieval-Augmented Generation

1.3.1 Khái niệm

Kiến trúc Retrieval-Augmented Generation được giới thiệu như một giải pháp để giải quyết triệt để vấn đề khi tiếp cận những kiến thức chuyên biệt. Thay vì chỉ dựa vào kiến thức "nội tại" mà LLM đã học được trong quá trình huấn luyện, RAG cho phép mô hình truy cập và sử dụng kiến thức từ một nguồn dữ liệu mà ta cung cấp ngay tại thời điểm trả lời. RAG là viết tắt của từ Retrieval-Augmented Generation, là một kiến trúc trong đó mô hình ngôn ngữ được "tăng cường" bằng thông tin được "truy xuất" từ một cơ sở kiến thức bên ngoài.



Hình 1.4: Kiến trúc RAG [3]

Nguyên lý hoạt động của RAG về cơ bản có thể chia làm 2 giai đoạn:

1. Truy xuất (Retrival)

- Khi người dùng đưa ra một câu hỏi (query), hệ thống không gửi câu hỏi này trực tiếp đến LLM.
- Thay vào đó, query đầu tiên được chuyển đến một bộ truy xuất (Retriever). Bộ truy xuất này có nhiệm vụ tìm kiếm trong một cơ sở kiến thức đã được chuẩn bị trước (ví dụ: tài liệu PDF của đồ án, văn bản pháp luật, tài liệu nội bộ...).
- Kết quả của giai đoạn này là một hoặc nhiều đoạn văn bản được đánh giá là có liên quan nhất đến câu hỏi của người dùng.

2. Tăng cường và tạo sinh (Augmented Generation)

- Hệ thống lấy các đoạn văn bản (context) vừa truy xuất được và "bổ sung" chúng vào câu hỏi gốc của người dùng.
- Quá trình này thường được thực hiện thông qua một mẫu câu lệnh (prompt template) được thiết kế đặc biệt. Ví dụ:
 - Prompt cũ (chỉ có LLM): "Trình bày về ưu điểm của RAG?"
 - Prompt mới (dùng RAG): "Dựa vào thông tin sau đây: [...đoạn văn bản A được truy xuất...] và [...đoạn văn bản B được truy xuất...]. Hãy trả lời câu hỏi: Trình bày về ưu điểm của RAG?"
- Câu lệnh mới, đã được "tăng cường" ngữ cảnh này, sau đó mới được gửi đến LLM. LLM sẽ sinh ra câu trả lời dựa trên thông tin được cung cấp, thay vì phải "tưởng tượng" ra câu trả lời từ kiến thức nội tại của nó.

1.3.2 Hạn chế

Mặc dù kiến trúc Retrieval-Augmented Generation (RAG) đã cải thiện đáng kể khả năng cung cấp thông tin chính xác của mô hình ngôn ngữ lớn bằng cách kết hợp với nguồn tri thức bên ngoài, phương pháp này vẫn tồn tại nhiều hạn chế quan trọng cần được xem xét.

- **Thiếu tính linh hoạt:** RAG truyền thống hoạt động theo một quy trình cố định gồm hai bước là: truy xuất và tạo sinh gây thiếu khả năng tự đánh giá và điều chỉnh trong quá trình suy luận. Nếu các tài liệu được truy xuất ban đầu không đủ liên quan hoặc không chứa thông tin cần thiết, hệ thống vẫn tiếp tục sử dụng các dữ liệu đó để sinh câu trả lời mà không có cơ chế phản hồi hay cải thiện truy vấn. Điều này dẫn đến việc câu trả lời cuối cùng có thể không chính xác hoặc không đầy đủ.
- **Phụ thuộc lớn vào hiệu quả của bộ truy xuất.** Nếu người dùng nhập truy vấn một cách mơ hồ hoặc sử dụng nhiều ngôn ngữ tự nhiên phức tạp sẽ khiến cho việc ánh xạ sang không gian embedding trở nên khó khăn. Khi đó, hệ thống có thể truy xuất các đoạn văn bản không thực sự liên quan, làm giảm độ chính xác mô hình.
- **Không có khả năng Multi-hop reasoning:** Đối với các câu hỏi cần tổng hợp thông tin từ nhiều nguồn khác nhau hoặc yêu cầu liên kết giữa các đoạn kiến thức, mô hình thường gặp khó khăn do chỉ thực hiện một lần truy xuất duy nhất. Điều này hạn chế khả năng xử lý các câu hỏi phức tạp, đặc biệt trong các lĩnh vực có cấu trúc thông tin chặt chẽ như văn bản pháp lý hay quy chế.
- **Không có cơ chế kiểm chứng hoặc đánh giá lại độ tin cậy của thông tin được truy xuất.** Các đoạn văn bản được đưa vào làm ngữ cảnh có thể chứa thông tin lỗi thời,

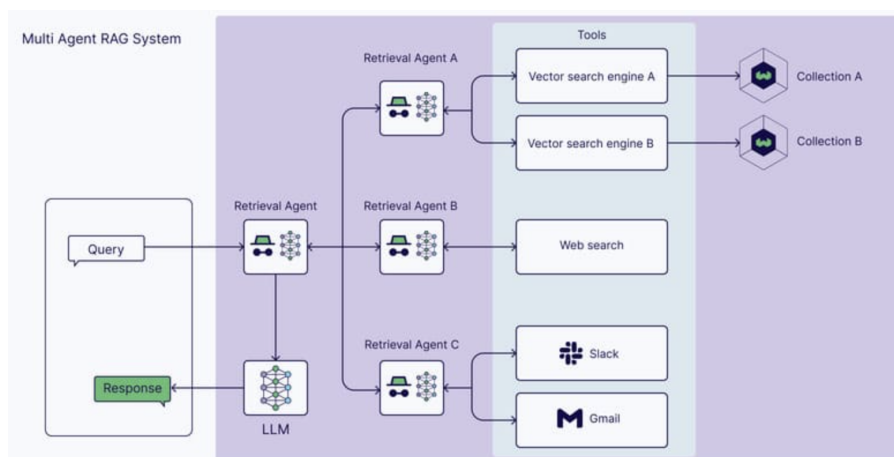
không đầy đủ hoặc không phù hợp với câu hỏi. Tuy nhiên, mô hình vẫn sử dụng các thông tin này để sinh câu trả lời, dẫn đến nguy cơ tạo ra các nội dung thiếu chính xác hoặc gây hiểu nhầm.

1.4 Agentic RAG

1.4.1 Khái niệm

Agentic RAG là một hướng tiếp cận nâng cao của kiến trúc Retrieval-Augmented Generation, trong đó hệ thống không chỉ đơn thuần thực hiện truy xuất và tạo sinh, mà còn được mở rộng với khả năng suy luận, lập kế hoạch và tự điều chỉnh thông qua cơ chế tác nhân (Agent).

Thay vì hoạt động theo một pipeline tuyến tính cố định, Agentic RAG vận hành dưới dạng một vòng lặp suy luận (reasoning loop), cho phép hệ thống đánh giá và cải thiện kết quả qua nhiều bước trung gian.



Hình 1.5: Kiến trúc Agentic RAG [6]

Bằng cách này, Agentic RAG không chỉ nâng cao độ chính xác của phần thông tin được truy xuất mà còn tối ưu hóa cách thông tin được sử dụng trong quá trình tạo nội dung. Phương pháp này đặc biệt hữu ích cho các bài toán phức tạp đòi hỏi vừa hiểu dữ liệu, vừa lập kế hoạch, suy luận nhiều bước.

Một số AI Agent trong Agentic RAG có thể kể đến như:

- **Routing Agents:** Chịu trách nhiệm xác định nguồn dữ liệu và công cụ phù hợp để xử lý truy vấn người dùng quyết định nguồn truy xuất phù hợp nhất và chọn pipeline tối ưu để tạo phản hồi.
- **Query Planning Agents:** Đóng vai trò như người điều phối, các agent này chia truy vấn phức tạp thành các bước nhỏ hơn và phân phối cho các agent khác xử lý. Sau đó, chúng tổng hợp các kết quả thu được để tạo ra phản hồi hoàn chỉnh. Cơ chế này – còn gọi là AI orchestration – đặc biệt hiệu quả với các truy vấn đa chiều hoặc có logic phân nhánh.
- **ReAct Agents (Reasoning + Acting):** Đây là một framework giúp các agent vừa lập luận, vừa hành động theo từng bước. ReAct Agents có thể xác định công cụ hoặc bước xử lý phù hợp dựa trên kết quả của các bước trước, từ đó xây dựng một chuỗi phản ứng linh hoạt và thích ứng tốt với thông tin mới.

1.4.2 Sự khác biệt giữa RAG và Agentic RAG

Sự khác biệt cốt lõi giữa RAG truyền thống và Agentic RAG nằm ở cách thức xử lý truy vấn và mức độ “thông minh” trong quá trình suy luận.

Đặc điểm	RAG Truyền thống	Agentic RAG
Quy trình	Tuyến tính (Truy xuất → Tạo sinh). Chỉ thực hiện một lần duy nhất.	Vòng lặp suy luận: Có khả năng đánh giá kết quả trung gian và thực hiện lại nếu cần.
Xử lý truy vấn	Phụ thuộc hoàn toàn vào câu hỏi ban đầu của người dùng.	Tối ưu hóa truy vấn: Có khả năng tự viết lại câu hỏi (Rewriter) để tìm kiếm hiệu quả hơn.
Kiểm soát chất lượng	Chấp nhận tất cả tài liệu truy xuất được, dễ bị nhiễu thông tin.	Cơ chế đánh giá (Grading): Lọc bỏ tài liệu kém chất lượng, chỉ giữ lại thông tin hữu ích.
Khả năng suy luận	Hạn chế với các câu hỏi phức tạp cần nhiều bước giải quyết.	Đa bước (Multi-step): Truy xuất và tổng hợp từ nhiều nguồn để trả lời các vấn đề phức tạp.
Kiến trúc	Đơn giản, dễ triển khai nhưng thiếu linh hoạt.	Phức tạp (dạng đồ thị tác nhân), nhưng linh hoạt và dễ mở rộng trong thực tế.

Bảng 1.1: So sánh đặc điểm giữa RAG Truyền thống và Agentic RAG

Tóm lại, Agentic RAG có thể được xem là một phiên bản nâng cao của RAG truyền thống, giúp khắc phục các hạn chế về truy xuất, suy luận và độ tin cậy. Việc áp dụng Agentic RAG trong bài toán chatbot hỏi đáp quy chế sinh viên hứa hẹn sẽ mang lại hiệu quả cao hơn trong việc cung cấp thông tin chính xác, đầy đủ và phù hợp với nhu cầu của người dùng.

1.5 Tech stack và các kỹ thuật sử dụng

1.5.1 Tech stack

Python

Python là ngôn ngữ lập trình chủ đạo được sử dụng xuyên suốt dự án, từ pipeline xử lý dữ liệu ngoại tuyến đến triển khai hệ thống Agent trực tuyến. Hệ sinh thái thư viện phong phú của Python — bao gồm `langchain`, `langgraph`, `chromadb`, `sentence-transformers` và `streamlit` — cho phép tích hợp toàn bộ các thành phần của kiến trúc Agentic RAG trong một môi trường phát triển thống nhất, không phụ thuộc vào hạ tầng đám mây.

LangGraph

LangGraph được sử dụng để xây dựng và điều phối đồ thị trạng thái (StateGraph) của hệ thống Agent với 6 node chức năng: *intent_gate*, *retrieve*, *evaluate*, *rewrite*, *generate* và *confidence_gate*. Không giống với chuỗi xử lý tuyến tính, LangGraph cho phép định nghĩa các cạnh có điều kiện (Conditional Edges) để Agent tự động phân nhánh sang vòng lặp viết lại truy vấn hoặc tiến thẳng đến bước sinh câu trả lời tùy theo kết quả đánh giá tài liệu ở mỗi hop.

LangSmith

LangSmith được tích hợp như công cụ quan sát và truy vết (Observability & Tracing) toàn bộ quá trình thực thi của đồ thị LangGraph. Thông qua dashboard LangSmith, mọi lần gọi node, nội dung đầu vào/đầu ra của từng công cụ, thời gian xử lý và số lần lặp (hop) đều được ghi nhận chi tiết theo từng phiên hội thoại. Tính năng này đóng vai trò thiết yếu trong quá trình thực nghiệm, giúp phân tích và gỡ lỗi hành vi suy luận của Agent một cách trực quan mà không cần can thiệp vào mã nguồn.

Chroma DB

ChromaDB được sử dụng làm cơ sở dữ liệu vector bền vững (persistent vector store), lưu trữ toàn bộ các vector nhúng 1024 chiều sinh ra từ mô hình BAAI/bge-m3 cùng với siêu dữ liệu đi kèm mỗi đoạn văn bản. Trong Pha Trực tuyến, ChromaDB phục vụ thao tác tìm kiếm tương đồng cosine (Cosine Similarity Search) theo yêu cầu của RetrieveTool, trả về tối đa top_k tài liệu vượt ngưỡng điểm tương đồng đã cấu hình, đảm bảo tính nhất quán và an toàn của cơ sở tri thức trong toàn bộ quá trình vận hành.

1.5.2 Các kỹ thuật sử dụng

Structure-Aware & Hierarchical Chunking

Thay vì phân tách văn bản theo độ dài ký tự cố định, phương pháp này khai thác cấu trúc logic của tài liệu quy chế theo thứ bậc Chương ($\rightarrow \#$) và Điều ($\rightarrow \#\#$). Hệ thống ưu tiên tách tại các ranh giới tiêu đề cấu trúc; chỉ khi một điều khoản vượt ngưỡng `chunk_size` = 1000 ký tự thì mới áp dụng tách theo kích thước với `chunk_overlap` = 200 ký tự để bảo toàn tính liên tục ngữ nghĩa. Cách tiếp cận này đảm bảo mỗi đoạn chunk phản ánh trọn vẹn nội dung của một điều khoản cụ thể, tránh tình trạng mất ngữ cảnh pháp lý khi bị cắt ngang.

Metadata Enrichment

Mỗi đoạn văn bản sau khi được phân tách sẽ được gắn kèm một bộ siêu dữ liệu có cấu trúc gồm sáu trường: `source_file` (tên văn bản PDF gốc), `doc_type` (loại văn bản), `effective_date` (ngày hiệu lực), `applicable_students` (đối tượng áp dụng, ví dụ: “ $\geq$ K68”), `chapter_title` và `article_title`. Bộ siêu dữ liệu này phục vụ hai mục đích: hỗ trợ hệ thống lọc tài liệu theo đối tượng áp dụng chính xác và cung cấp thông tin trích dẫn nguồn minh bạch trong câu trả lời cuối cùng.

Conditional Multi-hop Retrieval

Thay vì thực hiện truy xuất một lần duy nhất theo kiến trúc RAG truyền thống, hệ thống triển khai vòng lặp truy xuất đa bước (Multi-hop) có điều kiện được điều phối bởi LangGraph. Sau mỗi lần truy xuất, `EvaluateTool` tính toán điểm tương đồng trung bình (`avg_sim`) và gọi LLM để đánh giá ngữ nghĩa nếu cần. Nếu tài liệu chưa đủ chất lượng, `RewriteTool` được kích hoạt để tạo ra truy vấn mới và vòng lặp tiếp tục cho đến khi đạt ngưỡng chất lượng hoặc vượt quá giới hạn `max_hops = 5`.

Query Rewriting

`RewriteTool` sử dụng LLM để chuyển đổi câu hỏi gốc của sinh viên — thường chứa từ lóng, viết tắt hoặc ngôn ngữ thông thường — thành truy vấn sử dụng thuật ngữ pháp lý chính xác hơn, phù hợp với ngôn ngữ của văn bản quy chế trong cơ sở tri thức. Ví dụ, cụm từ “bị đuổi học” sẽ được viết lại thành “buộc thôi học do số tín chỉ không đạt vượt ngưỡng”. Truy vấn mới được cập nhật vào `current_query` của `GraphState` và kết quả tìm kiếm từ vòng lặp trước bị xóa để tránh nhiễu, đảm bảo vòng lặp tiếp theo tìm kiếm với trạng thái sạch.

Confidence Scoring

Trước khi hiển thị câu trả lời cho người dùng, `ConfidenceGate` tính điểm tin cậy tổng hợp từ ba yếu tố độc lập: tỉ lệ tài liệu truy xuất được so với `top_k` (tối đa 30%), mức độ cụ thể của câu trả lời đo bằng sự xuất hiện của các pattern trích dẫn pháp lý như “Điều X”, “Khoản Y”, con số định lượng (tối đa 50%), và hiệu quả suy luận đo bằng số hop đã thực hiện (tối đa 20%). Dựa trên tổng điểm, hệ thống phân loại đầu ra thành ba trạng thái: *pass* ($\geq 65\%$), *warn* (35–65%) và *reject* ($< 35\%$), ngăn chặn hiệu quả hiện tượng hallucination trước khi câu trả lời đến tay sinh viên.

Chương 2

Phân tích và thiết kế hệ thống

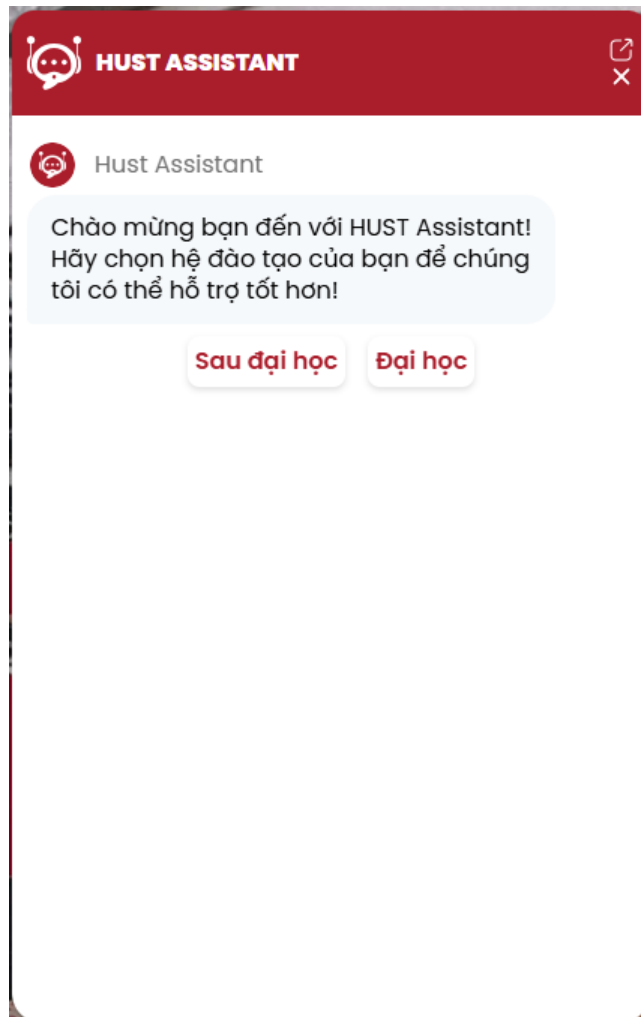
2.1 Khảo sát thực tế

Sự bùng nổ của trí tuệ nhân tạo, đặc biệt là các mô hình ngôn ngữ lớn và trí tuệ nhân tạo tạo sinh, đang tạo ra làn sóng chuyển đổi số mạnh mẽ trong phương thức quản trị và giảng dạy tại các cơ sở giáo dục đại học. Việc tích hợp hệ thống Chatbot vào công tác quản lý học vụ, tư vấn tuyển sinh và hỗ trợ sinh viên đang chuyển dần từ giai đoạn thử nghiệm sang triển khai thực tiễn. Nhằm xây dựng một hệ thống hỏi đáp quy chế sinh viên ứng dụng kiến trúc Agentic RAG đạt hiệu quả cao, việc tiến hành khảo sát thực tế là bước tiên quyết để định vị hiện trạng công nghệ, phân tích nhu cầu người dùng và nhận diện các lỗ hổng kỹ thuật cần khắc phục.

2.1.1 Hệ thống chatbot hiện tại

Việc khảo sát các hệ thống Chatbot đã và đang được triển khai cung cấp một bức tranh thực chứng về những kiến trúc đang thống trị trong khối giáo dục. Nhìn chung, các hệ thống này được phân loại thành hai thể hệ chính như đã trình bày ở trên: Chatbot dựa trên luật với các kịch bản tĩnh, và Chatbot dựa trên AI tạo sinh với khả năng xử lý ngôn ngữ tự nhiên năng động.

Tại Đại học Bách khoa Hà Nội, hệ thống Chatbot HUST trên trang quản lý đào tạo đã được triển khai để giải đáp các thắc mắc về hoạt động ngoại khóa, điểm rèn luyện và tuyển dụng, tập trung xử lý câu hỏi lặp lại từ hơn 35.000 sinh viên trong các đợt cao điểm. Song song đó, Viện Công nghệ Thông tin và Truyền thông (SoICT) cũng vận hành các trợ lý ảo hướng dẫn tuyển sinh chuyên biệt.



Hình 2.1: Hệ thống chatbot của trường

2.1.2 Ưu nhược điểm của hệ thống hiện tại

Việc đánh giá chuyên sâu các khía cạnh kỹ thuật của các thể hệ chatbot là cơ sở để thiết kế một hệ thống Agentic RAG an toàn và hiệu quả.

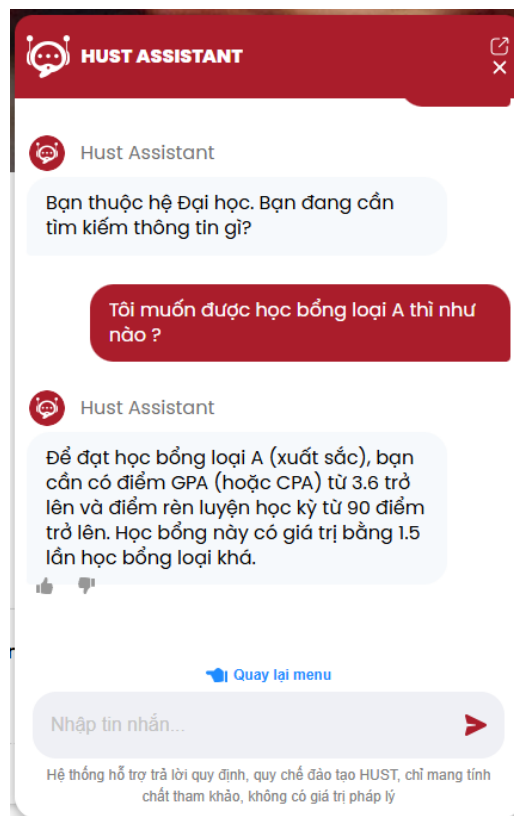
Ưu điểm

- Về tốc độ phản hồi, Chatbot cung cấp câu trả lời gần như tức thời và hoạt động liên tục 24/7, giúp xóa bỏ hoàn toàn sự phụ thuộc vào giờ hành chính của cán bộ quản lý, đặc biệt hữu ích trong các đợt cao điểm đăng ký học phần hay nộp hồ sơ xét học bổng.
- Về trải nghiệm người dùng, các hệ thống thể hệ mới được thiết kế với giao diện trực quan, bố cục trình bày rõ ràng và khả năng điều hướng thông minh.
- Hệ thống tự động phân loại câu hỏi của sinh viên sang các nhóm chủ đề đúng (học vụ, học bổng, tuyển sinh) và dẫn đến nguồn thông tin phù hợp chỉ trong một vài bước tương tác.

Tuy nhiên, quá trình khảo sát thực tế cho thấy các hệ thống hiện hành vẫn tồn tại những hạn chế căn bản, chưa đáp ứng được đặc thù của môi trường học vụ đại học tín chỉ.

Nhược điểm

- **Thiếu cơ chế đối thoại thu thập thông tin:** Hệ thống hiện tại xử lý mỗi câu hỏi như một truy vấn độc lập, đơn chiều. Khi sinh viên đặt một câu hỏi chưa đủ thông tin đặc trưng (ví dụ: chỉ hỏi "chuẩn ngoại ngữ đầu ra là bao nhiêu?" mà không kèm thông tin ngành học và mã khóa), hệ thống hoặc trả về một câu trả lời chung chung không áp dụng được, hoặc từ chối hoàn toàn.
- **Thiếu khả năng truy xuất thông tin sâu theo ngữ cảnh:** Hệ thống hiện hành chủ yếu hoạt động theo mô hình điều hướng nếu gặp thông tin không có trong tri thức. Khi sinh viên hỏi về tín chỉ học phí của một ngành, hệ thống sẽ điều hướng tới một trang để sinh viên tự giải đáp. Cách tiếp cận này chưa giải quyết được nhu cầu thực sự: sinh viên cần biết mức học phí cụ thể cho ngành học và hệ đào tạo của chính họ, không phải một tập hợp bảng giá áp dụng cho toàn trường.
- **Chưa có khả năng truy vết thông tin:** Hệ thống hiện tại có thể trả lời các câu hỏi mà sinh viên đưa ra nhưng chưa đưa ra được nguồn dẫn chứng giúp người dùng có thể tự mình kiểm chứng lại thông tin



Hình 2.2: Hệ thống chưa có khả năng truy vết

2.2 Phân tích bài toán và yêu cầu hệ thống

2.2.1 Phân tích bài toán

Việc xây dựng một hệ thống chatbot hỗ trợ sinh viên không chỉ đơn thuần là bài toán kỹ thuật mà còn là bài toán giải quyết nhu cầu thực tiễn trong công tác quản lý và hỗ trợ đào

tạo. Hệ thống hướng tới hai nhóm đối tượng chính: nhóm người dùng cuối (sinh viên) và nhóm quản lý (Đại học)

- Đối với nhóm sinh viên: Đây là nhóm đối tượng sử dụng chính với số lượng lớn và nhu cầu tra cứu thông tin thường xuyên. Các vấn đề thực tế sinh viên đang gặp phải bao gồm:
 - **Khó khăn trong tiếp cận thông tin:** Có rất nhiều văn bản quy chế khác nhau, không những thế đối với mỗi quy chế lại sẽ chỉ áp dụng cho mỗi nhóm sinh viên khác nhau. Ví dụ như quy định đầu ra ngoại ngữ của chương trình đào tạo thường sẽ khác với chương trình tiên tiến hoặc chương trình liên kết,... Việc tìm kiếm thông tin phù hợp đối tượng bản thân sẽ gây mất nhiều thời gian
 - **Rào cản về ngôn ngữ hành chính:** Các văn bản quy phạm pháp luật thường sử dụng ngôn ngữ hành chính trang trọng, câu từ dài và có tính liên kết cao. Sinh viên, đặc biệt là sinh viên năm nhất, thường gặp khó khăn trong việc hiểu đúng và đủ ý nghĩa của các quy định.
 - **Nhu cầu phản hồi tức thì:** Sinh viên thường có nhu cầu giải đáp thắc mắc bất kể thời gian, đặc biệt là trong các giai đoạn cao điểm như đăng ký tín chỉ, xét tốt nghiệp hay nộp hồ sơ xét học bổng. Các kênh hỗ trợ truyền thống thường bị quá tải và có độ trễ lớn.
- Đối với Đại học và bộ phận hỗ trợ
 - **Giảm tải áp lực hành chính:** Bộ phận hành chính và cố vấn học tập thường xuyên phải trả lời lặp đi lặp lại những câu hỏi có nội dung giống nhau từ hàng nghìn sinh viên. Điều này gây lãng phí nguồn nhân lực chất lượng cao cho các tác vụ đơn giản.
 - **Đồng bộ hóa thông tin:** Đảm bảo mọi sinh viên đều nhận được một câu trả lời thống nhất cho cùng một vấn đề, tránh tình trạng "tam sao thất bản" khi thông tin được truyền miệng giữa các sinh viên.

2.2.2 Yêu cầu chức năng

Dựa trên mục tiêu xây dựng hệ thống chatbot hỏi đáp quy chế sinh viên ứng dụng kiến trúc AgenticRAG, hệ thống cần đáp ứng các yêu cầu chức năng sau:

- **Tiếp nhận và xử lý câu hỏi của người dùng:** Hệ thống cho phép người dùng nhập câu hỏi bằng ngôn ngữ tự nhiên thông qua giao diện web. Câu hỏi có thể được đặt dưới nhiều hình thức khác nhau, bao gồm câu hỏi trực tiếp, câu hỏi có từ viết tắt hoặc câu hỏi mang tính hội thoại.
- **Phân loại ý định truy vấn:** Hệ thống có khả năng xác định loại câu hỏi mà người dùng đang yêu cầu, chẳng hạn như câu hỏi về học vụ, học phí, đăng ký môn học, điều kiện tốt nghiệp hoặc các quy định khác liên quan đến sinh viên. Kết quả phân loại được sử dụng để định hướng quá trình truy xuất và suy luận.
- **Hỗ trợ truy xuất đa bước:** Đối với các câu hỏi phức tạp yêu cầu tổng hợp thông tin từ nhiều nguồn khác nhau, hệ thống phải có khả năng thực hiện nhiều vòng truy xuất và suy luận để thu thập đầy đủ ngữ cảnh cần thiết trước khi tạo câu trả lời.

- **Sinh câu trả lời dựa trên ngữ cảnh truy xuất:** Hệ thống sử dụng mô hình ngôn ngữ lớn để tạo câu trả lời dựa trên các tài liệu đã được truy xuất. Nội dung phản hồi phải bám sát nguồn dữ liệu và hạn chế tối đa hiện tượng sinh thông tin không có trong tài liệu.
- **Hỗ trợ hội thoại đa lượt:** Hệ thống cần lưu trữ ngữ cảnh hội thoại để xử lý các câu hỏi tiếp theo có liên quan đến nội dung đã trao đổi trước đó.
- **Quản lý cơ sở tri thức:** Hệ thống cho phép cập nhật, bổ sung hoặc thay thế các tài liệu quy chế sinh viên khi có phiên bản mới mà không làm ảnh hưởng đến quá trình vận hành của chatbot.

2.2.3 Yêu cầu phi chức năng

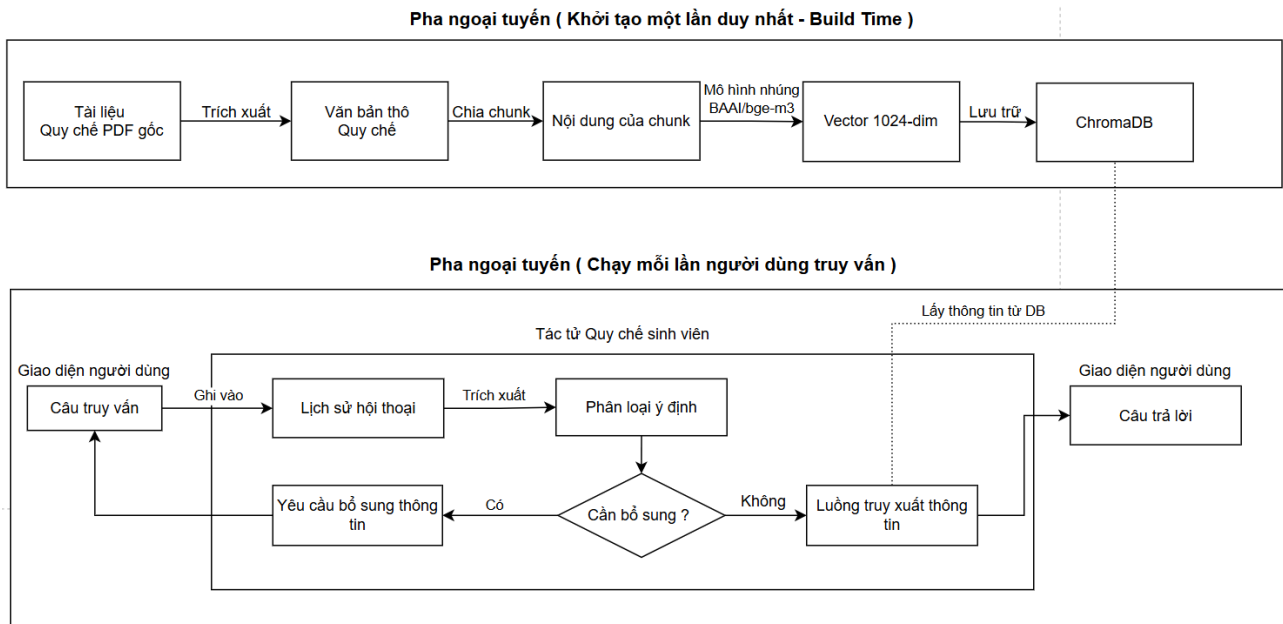
Bên cạnh các yêu cầu chức năng, hệ thống cũng cần đáp ứng các yêu cầu phi chức năng nhằm đảm bảo hiệu quả vận hành, khả năng mở rộng và chất lượng dịch vụ.

- **Hiệu năng:** Hệ thống cần đảm bảo thời gian phản hồi ngắn để mang lại trải nghiệm tốt cho người dùng.
- **Độ chính xác:** Các câu trả lời được sinh ra phải có độ chính xác cao, bám sát nội dung của các tài liệu quy chế và hạn chế tối đa hiện tượng hallucination của mô hình ngôn ngữ lớn.
- **Khả năng giám sát:** Hệ thống cần hỗ trợ ghi log và theo dõi hoạt động nhằm phục vụ công tác kiểm tra, đánh giá và khắc phục sự cố trong quá trình vận hành.
- **Khả năng giải thích:** Các câu trả lời được sinh ra cần đi kèm nguồn tham khảo để người dùng có thể kiểm chứng và đối chiếu với văn bản quy chế gốc.
- **Khả năng bảo trì:** Mã nguồn và các thành phần của hệ thống cần được tổ chức theo hướng mô-đun hóa nhằm tạo điều kiện thuận lợi cho việc bảo trì, nâng cấp và sửa lỗi trong tương lai.

2.3 Kiến trúc tổng thể hệ thống

2.3.1 Sơ đồ kiến trúc tổng thể

Toàn bộ hệ thống được chia thành hai pha hoạt động độc lập: pha ngoại tuyến - chạy một lần khi xây dựng cơ sở tri thức hoặc muốn cập nhật chỉnh sửa cơ sở tri thức và pha trực tuyến - xử lý câu hỏi thời gian thực và chạy theo từng yêu cầu.



Hình 2.3: Sơ đồ kiến trúc tổng thể

- **Pha ngoại tuyến – Xây dựng cơ sở tri thức:** Pha ngoại tuyến được thực hiện một lần hoặc khi xuất hiện tài liệu mới cần cập nhật vào hệ thống. Mục tiêu chính của giai đoạn này là chuyển đổi các văn bản PDF thô thành cơ sở dữ liệu vector có khả năng tìm kiếm ngữ nghĩa rồi lưu trữ trong cơ sở dữ liệu vector.
- **Pha ngoại tuyến – Xử lý truy vấn thời gian thực:** Pha ngoại tuyến sẽ được kích hoạt mỗi khi người dùng gửi câu hỏi đến hệ thống, được thiết kế theo nguyên lý "Phân tách trách nhiệm" với ba tầng hoạt động độc lập. Việc cấu hình ba tầng này giúp tách biệt rạch ròi giữa phần "kiểm duyệt", phần "suy luận cốt lõi" và phần "lưu trữ", giúp hệ thống dễ bảo trì, dễ thay đổi mô hình LLM, và đặc biệt là kiểm soát chặt chẽ hiện tượng "ảo giác".
 - **Tầng xử lý dữ liệu:** Tầng này đóng vai trò là "cửa ngõ" bảo vệ hệ thống khỏi các đầu vào sai lệch và ngăn chặn đầu ra bị "ảo giác".
 - **Tầng suy luận tác tử:** Tập trung vào việc truy xuất và đưa ra câu trả lời chính xác nhất
 - **Tầng bộ nhớ:** Quản lý ngữ cảnh theo từng phiên hội thoại.

2.4 Thiết kế cơ sở dữ liệu

2.4.1 Khảo sát và đặc tả dữ liệu nguồn

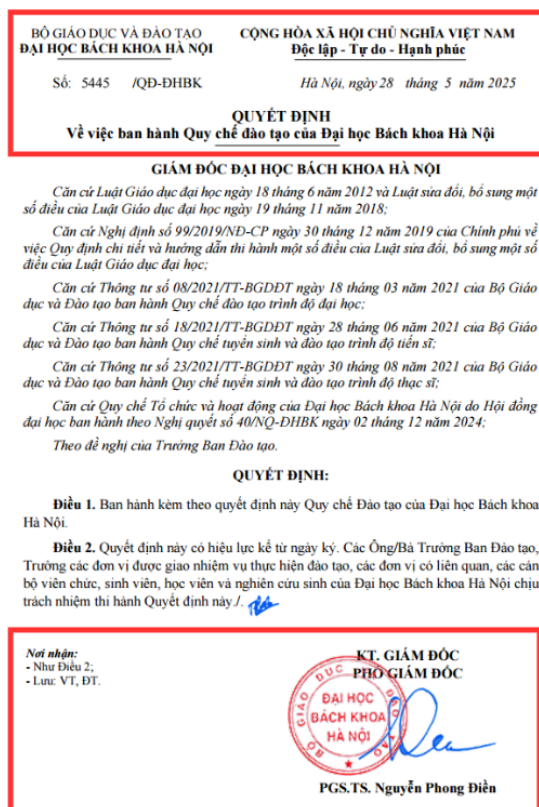
Nguồn dữ liệu thu thập

Dữ liệu được thu thập chủ yếu từ các văn bản chính thống do Đại học Bách Khoa ban hành bao gồm nhưng không giới hạn:

- Các quy định và biểu mẫu thường dùng được thu thập từ trên trang "Sổ tay sinh viên Bách Khoa". Nội dung sẽ bao gồm các thông tin về đào tạo, công tác sinh viên, các văn bản hướng dẫn đại học và sau đại học.

Phân tích bộ dữ liệu

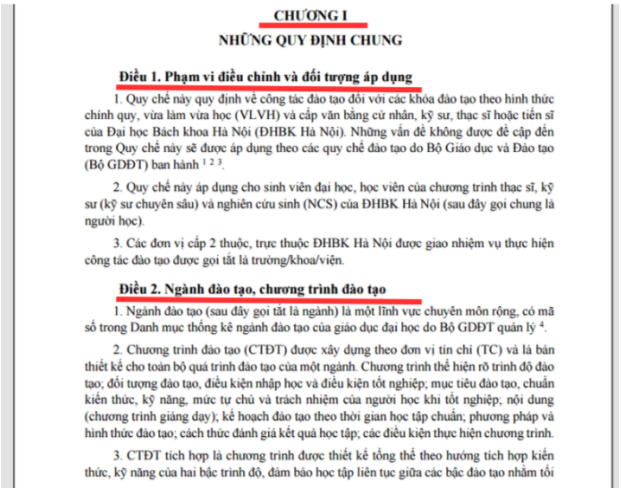
- **Định dạng dữ liệu:** Hầu hết tài liệu tồn tại dưới dạng PDF, thường chứa nhiều "nhiều" như tiêu đề trang, số trang,... Gây khó khăn cho việc trích xuất văn bản thuần túy.



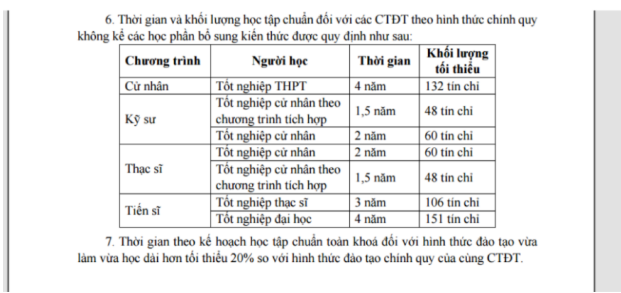
Hình 2.4: Tài liệu có nhiều thông tin nhiễu

- **Cấu trúc dữ liệu:** Các văn bản quy chế có cấu trúc phân cấp chặt chẽ: Chương -> Điều -> Mục. Nếu thực hiện kỹ thuật cắt đoạn một cách máy móc theo số lượng ký tự, rất dễ làm gãy cấu trúc ngữ nghĩa. Ví dụ: Một "Điều" bị cắt đôi sẽ làm mất ngữ cảnh. Do đó, yêu cầu hệ thống phải có chiến lược cắt dựa trên cấu trúc văn bản hoặc ngữ nghĩa.

- **Dữ liệu dạng bảng biểu:** Một số thông tin được trình bày dưới dạng bảng, đòi hỏi phải có bước tiền xử lý đối với dữ liệu bảng.

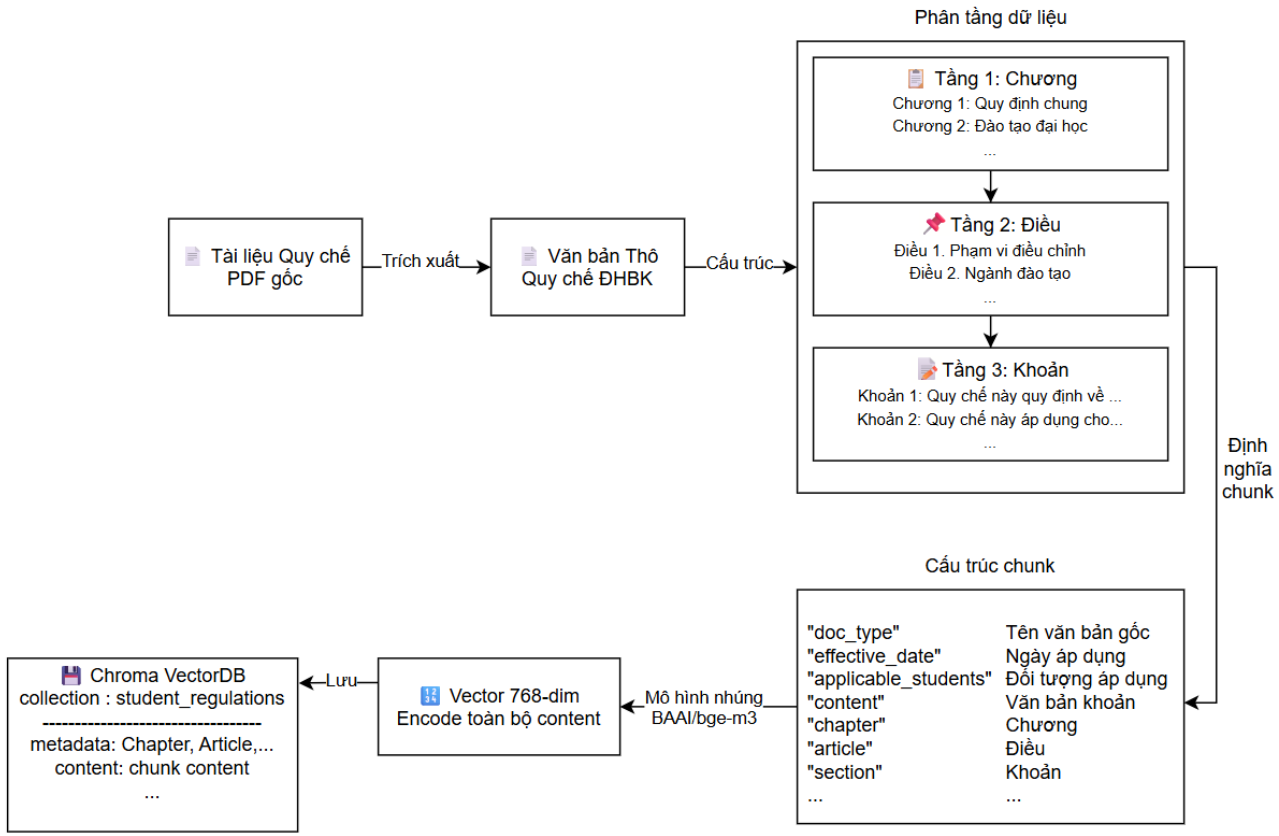


Hình 2.5: Dữ liệu có cấu trúc chặt chẽ



Hình 2.6: Dữ liệu dạng bảng

2.4.2 Xây dựng cơ sở tri thức



Hình 2.7: Cấu trúc phân tầng dữ liệu

Hệ thống cần xử lý tài liệu quy chế dưới dạng PDF thô thành dữ liệu có cấu trúc, có thể truy vết nguồn và hỗ trợ suy luận của Agent. Quy trình gồm các bước chính:

1. Trích xuất văn bản gốc:

Sử dụng thư viện `pdfplumber` để đọc nội dung từng trang trong file PDF. Văn bản thuần và dữ liệu bảng được trích xuất độc lập nhằm đảm bảo không làm mất thông tin bảng biểu.

2. Phân tích cấu trúc tài liệu:

- Xác định các thành phần không mang giá trị ngữ nghĩa như header, footer,... Chuẩn hóa khoảng trắng và định dạng lại nhằm đảm bảo tính nhất quán.
- Nhận diện được cấp độ phân cấp của tài liệu quy chế dựa theo cấu hình `regex` để tách được các Chương -> Điều -> Khoản.
- Áp dụng Hierarchical Chunking theo cấu trúc tự nhiên của tài liệu (không cắt ngẫu nhiên dựa trên số ký tự) để đảm bảo được tính cấu trúc logic rõ ràng và toàn vẹn của tài liệu.

3. Cấu hình cho chunk và gắn metadata:

- Dữ liệu được chia theo các cấp nội dung "Chương" và "Điều". Trong trường hợp một "Điều" có độ dài lớn, hệ thống tiếp tục chia nhỏ bằng phương pháp chunking với kích thước giới hạn (`chunk_size`) là 1024 token và trùng lặp (`overlap`) 256 token .
- Đối với dữ liệu dạng bảng, hệ thống lưu trữ toàn bộ bảng trong một chunk riêng biệt. Mỗi đoạn dữ liệu sau khi chunking đều được gắn metadata đầy đủ, bao gồm thông tin văn bản gốc, chương, điều và cờ nhận diện dữ liệu bảng.

4. Lưu trữ gian dạng JSON:

Dữ liệu được lưu trữ dưới dạng các đối tượng JSON. Mỗi đối tượng bao gồm phần nội dung văn bản đã xử lý và hệ metadata tương ứng như loại văn bản, thời điểm hiệu lực và đối tượng áp dụng.

5. Nhúng văn bản và lưu trữ trong VectorDB:

- Sử dụng mô hình nhúng `BAAI/bge-m3` để tạo ra vector embedding 1024 chiều cho mỗi đoạn chunk.
- Các chunk sau khi được chuyển đổi thành vector sẽ được lưu trữ trong cơ sở dữ liệu vector ChromaDB với tên là `student_regulations`

2.5 Thiết kế tầng xử lý dữ liệu

Trong kiến trúc tổng thể của hệ thống AgenticRAG, tầng xử lý dữ liệu đóng vai trò là lớp bảo vệ bao bọc xung quanh tầng suy luận agent. Sự phân tách này xuất phát từ một quan sát thực tiễn trong thiết kế hệ thống hỏi-đáp: hai nguồn sai lệch chính ảnh hưởng đến chất lượng kết quả không nằm ở bản thân quá trình suy luận, mà nằm ở đầu vào chưa đầy đủ (câu hỏi thiếu thông tin đặc tả) và đầu ra không đáng tin cậy (câu trả lời tổng hợp từ ngữ liệu kém chất lượng). Tầng này triển khai hai cơ chế kiểm soát tương ứng: **Cổng tiền xử lý - Intent Gate** và **Cổng hậu xử lý - Confidence Gate**.

2.5.1 Cổng tiền xử lý

Các câu hỏi của sinh viên trong thực tế thường không đầy đủ thông tin hoặc mang tính mơ hồ theo ngữ cảnh. Câu hỏi như *"TOEIC bao nhiêu thì đủ ra trường?"* hoàn toàn hợp lệ về mặt ngữ pháp nhưng lại thiếu thông tin về khóa học của sinh viên vì chuẩn ngoại ngữ đầu ra tại Đại học Bách khoa Hà Nội khác nhau tùy theo khóa học. Nếu để Agent trực tiếp truy vấn cơ sở dữ liệu vector với câu hỏi như vậy, kết quả thu về sẽ không xác định hoặc trả về thông tin không phù hợp với đối tượng người dùng cụ thể. Cổng tiền xử lý được thiết kế để giải quyết vấn đề này.

Cổng tiền xử lý dữ liệu sẽ sử dụng kiến trúc phân loại ý định kết hợp (Hybrid Intent Classification), kiến trúc này sẽ bao gồm ba tầng lần lượt là: Trích xuất đặc trưng bằng LLM-Hợp nhất thực thể theo ngữ cảnh-Đối chiếu tập luật nghiệp vụ. Trước khi đi vào cấu trúc cụ thể từng phần, ta sẽ cần thiết lập lược đồ ý định (Schema Intent) giúp xác định xem những ý định nào sẽ cần phải hỏi lại thông tin từ người dùng.

Thiết lập lược đồ ý định:

Hệ thống phân rã các yêu cầu của sinh viên thành một tập hợp các ý định được định nghĩa trước. Lược đồ này quy định rõ tính phụ thuộc của từng ý định đối với các thực thể đi kèm. Toàn bộ dữ liệu quy chế sinh viên của Đại học Bách khoa Hà Nội sẽ được chia thành các nhóm ý định như sau:

- **Nhóm ý định độc lập:** Bao gồm các quy chế chung áp dụng cho toàn bộ sinh viên. Nhóm này không yêu cầu trích xuất thực thể và được chuyển tiếp trực tiếp đến tác tử quy chế sinh viên.
- **Nhóm ý định phụ thuộc:** Bao gồm các quy định đặc thù như yêu cầu ngoại ngữ hoặc học phí. Các ý định này bị ràng buộc nghiêm ngặt và bắt buộc phải có đủ các tham số thực thể (ví dụ: ngành học, khóa học) mới đủ điều kiện truy xuất.

```
intents:
  GENERAL_REGULATION:
    description: "Quy chế, quy định chung không phụ thuộc ngành/khóa cụ thể"
    requires_entities: false # Không cần entity → pass thẳng vào RAG
    required_fields: []
    examples:
      - "GPA bao nhiêu thì bị cảnh cáo học vụ?"
      - "Điều kiện bảo lưu kết quả học tập?"

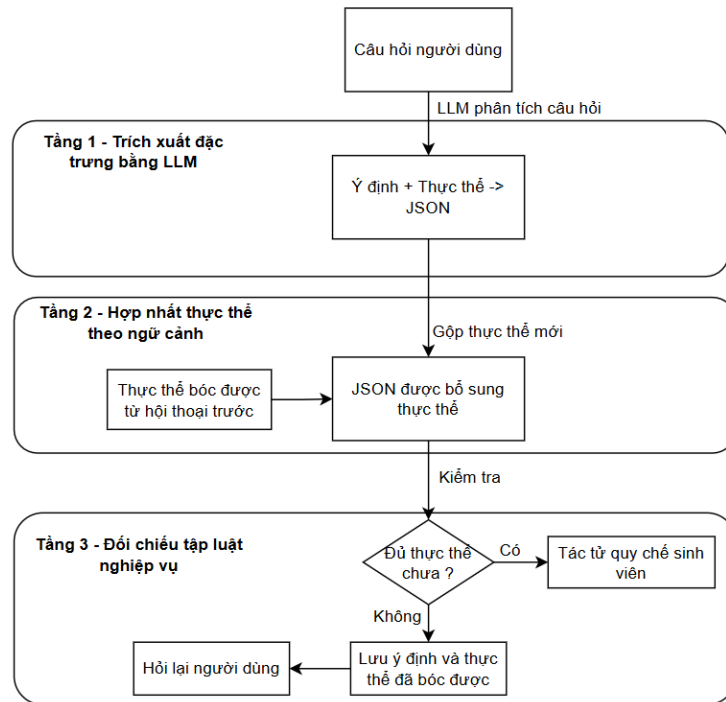
  LANGUAGE_REQUIREMENT:
    description: "Yêu cầu ngoại ngữ đầu vào, chuẩn đầu ra theo ngành và khóa"
    requires_entities: true # BẮT BUỘC có entity
    required_fields:
      - ngành_hoc # "CNTT", "Cơ điện tử", "Toán - Tin"
      - khoa_hoc # "K65", "K68", "K70"
    clarification_template: |
      Để tra cứu yêu cầu ngoại ngữ chính xác, bạn vui lòng cho biết:
      - Ngành học của bạn là gì? (ví dụ: CNTT, Cơ điện tử, Toán - Tin...)
      - Bạn thuộc khóa nào? (ví dụ: K65, K68, K70...)

  TUITION_FEE:
    description: "Học phí, miễn giảm học phí, quy định đóng học phí"
    requires_entities: true
    required_fields: [ngành_hoc, khoa_hoc]
    clarification_template: "Để tra cứu học phí chính xác..."
```

Hình 2.8: Intent Schema

Cơ chế phân loại ba tầng

Quá trình phân loại và chuẩn bị tham số được hiện thực hóa qua ba tầng xử lý nối tiếp:



Hình 2.9: Luồng phân loại Intent ba tầng

• Tầng 1: Trích xuất đặc trưng bằng LLM

Hệ thống sử dụng kỹ thuật Prompt Engineering để cung cấp cho LLM lịch sử hội thoại và câu hỏi hiện tại. LLM làm nhiệm vụ bóc tách câu hỏi thành một cấu trúc dữ liệu JSON định dạng sẵn gọi là tập thực thể.

```
_INTENT_EXTRACTION_PROMPT = """\
Bạn là bộ phân loại câu hỏi của hệ thống chatbot quy chế đào tạo ĐHBK Hà Nội.

## Danh sách Intent hợp lệ:
{intent_list}

## Lịch sử hội thoại gần đây:
{memory_context}

## Câu hỏi hiện tại:
"{question}"

## Nhiệm vụ:
1. Xác định intent phù hợp nhất
2. Bóc tách entity: ngành_hoc, khoa_hoc, gpa

Trả về JSON:
{"intent": "INTENT_NAME", "entities": {...}, "confidence": 0.0-1.0}
"""
```

Câu hỏi: "Sinh viên Cơ Điện tử K68 cần TOEIC bao nhiêu?"

```
→ LLM output: {
  "intent": "LANGUAGE_REQUIREMENT",
  "entities": {"ngành_hoc": "Cơ điện tử", "khoa_hoc": "K68"},
  "confidence": 0.95
}
```

Hình 2.10: Tầng 1: LLM Extraction

- **Tầng 2: Hợp nhất thực thể theo ngữ cảnh**

Hệ thống sẽ truy xuất lịch sử hội thoại, lấy ra các thực thể (`memory_entities`) và ý định đã trích xuất (`previous_intent`) được từ hội thoại trước. Sau đó hệ thống kiểm tra từng trường thực thể bắt buộc theo lược đồ. Nếu một trường còn trống nhưng tồn tại trong `memory_entities`, hệ thống tự động điền giá trị từ bộ nhớ mà không cần hỏi lại người dùng. Đây là cơ chế then chốt giúp hội thoại diễn ra tự nhiên trong các phiên đa lượt.

- **Tầng 3: Đối chiếu tập luật nghiệp vụ**

Tập thực thể sau đó được đối chiếu với lược đồ ý định đã định nghĩa bằng YAML. Tầng này sử dụng logic để kiểm tra sự toàn vẹn của dữ liệu. Nếu ý định thuộc nhóm phụ thuộc nhưng trong tập thực thể bị khuyết thiếu trường thông tin bắt buộc, hệ thống sẽ kích hoạt luồng làm rõ.

- Nếu đủ thông tin → trả về `IntentResult` với `needs_clarification = False`, chuyển câu hỏi và thực thể sang tầng suy luận tác tử.
- Nếu thiếu thông tin → trả về `IntentResult` với `needs_clarification = True`, lưu lượt hỏi làm rõ vào lịch sử hội thoại và phản hồi câu hỏi làm rõ thay cho câu trả lời thực chất.

2.5.2 Cổng hậu xử lý

Một trong những rủi ro đặc thù của hệ thống RAG là hiện tượng "ảo giác" (Hallucination)- là khi mô hình ngôn ngữ lớn tổng hợp ra câu trả lời nghe có vẻ chính xác nhưng thực tế không có căn cứ trong tài liệu nguồn. Trong bài toán hỏi đáp quy chế sinh viên thì việc kiểm soát ảo giác là bắt buộc cần thiết. Từ đó cổng hậu xử lý sẽ xử lý vấn đề này.

Thuật toán tính điểm tin cậy

Thay vì dựa hoàn toàn vào một chỉ số đơn lẻ, hệ thống tính toán điểm tổng hợp từ ba yếu tố độc lập, mỗi yếu tố phản ánh một khía cạnh khác nhau của chất lượng kết quả:

- **Yếu tố 1 - Chất lượng truy xuất (tối đa 30%):** Đánh giá dựa trên số lượng tài liệu liên quan thu thập được trong toàn bộ quá trình suy luận. Không có tài liệu nào đồng nghĩa với 0%, từ ba tài liệu trở lên đạt mức tối đa 30%. Yếu tố này đo lường mức độ phủ của cơ sở tri thức đối với câu hỏi được đặt ra.
- **Yếu tố 2 - Chất lượng tạo sinh (tối đa 40%):** Đánh giá tính cụ thể và căn cứ của câu trả lời thô. Hệ thống kiểm tra hai tiêu chí:
 - Câu trả lời có chứa các con số, tỷ lệ hoặc thời hạn cụ thể không (ví dụ: "4.5", "75%", "8 học kỳ").
 - Câu trả lời có trích dẫn rõ ràng điều khoản pháp lý không (ví dụ: "Điều 15", "Khoản 3").

Mỗi tiêu chí đóng góp tối đa 20%, phản ánh mức độ câu trả lời thực sự được căn cứ vào tài liệu chứ không phải suy diễn tự do của LLM. Nếu câu trả lời chứa những từ ngữ phủ định như "không biết", "không tìm thấy", "không có thông tin"... thì hệ thống sẽ chấm 0 cho yếu tố này. Ngoài ra để câu trả lời vừa không chứa các con số, vừa không trích dẫn rõ ràng điều khoản nhưng cũng đồng thời không chứa các từ phủ định thì vẫn sẽ được 10%.

- **Yếu tố 3 - Hiệu suất suy luận (tối đa 30%):** Đánh giá gián tiếp thông qua số vòng lặp Agent đã phải tiêu tốn. Câu trả lời tìm được ở vòng đầu tiên ($hop = 1$) nhận điểm tối đa 30%; mỗi vòng lặp thêm làm giảm điểm, phản ánh thực tế rằng số vòng lặp nhiều hơn thường cho thấy tài liệu không khớp tốt với câu hỏi ban đầu.

Cơ chế phân loại và hành động

Dựa trên điểm tin cậy, cổng hậu xử lý phân loại kết quả vào một trong ba mức và thực hiện hành động tương ứng:

- **PASS ($\geq 65\%$):** Câu trả lời được chấp nhận và truyền nguyên vẹn lên giao diện người dùng cùng với danh sách nguồn trích dẫn và điểm tin cậy hiển thị dưới dạng badge màu xanh lá.
- **WARN ($35\% - 65\%$):** Câu trả lời được giữ nguyên nội dung nhưng được đính kèm một cảnh báo tiêu chuẩn thông báo về mức độ tin cậy trung bình và khuyến nghị sinh viên xác minh lại với phòng ban có thẩm quyền. Badge hiển thị màu vàng.
- **REJECT ($\leq 35\%$):** Hệ thống từ chối trả về nội dung do Agent tổng hợp và thay thế bằng một thông báo mặc định thông báo không tìm thấy thông tin chính xác, đồng thời cung cấp các gợi ý: từ khóa tìm kiếm thay thế, địa chỉ website chính thức và hướng liên hệ phòng ban phụ trách. Đây là biện pháp bảo vệ cuối cùng chống lại "ảo giác".

Sau khi thực hiện hành động phân loại, cổng hậu xử lý ghi lượt hội thoại vào cửa sổ ngữ cảnh để phục vụ các câu hỏi tiếp theo, hoàn tất một chu trình xử lý hoàn chỉnh của hệ thống.

2.6 Thiết kế tầng suy luận tác tử

Tầng suy luận tác tử là thành phần trung tâm của hệ thống, chịu trách nhiệm thực thi toàn bộ quá trình tìm kiếm, đánh giá và tổng hợp thông tin để tạo ra câu trả lời cuối cùng. Tầng này tiếp nhận đầu vào từ cổng tiền xử lý của tầng xử lý dữ liệu - bao gồm câu hỏi đã được làm sạch, phân loại ý định và đầy đủ thực thể - và trả về câu trả lời thô (raw answer) cổng hậu xử lý của tầng xử lý dữ liệu kiểm duyệt. Nhờ được bao bọc bởi hai cổng kiểm soát, tầng này được giải phóng hoàn toàn khỏi các tác vụ xử lý ngoại lệ, cho phép tập trung tối đa vào chất lượng suy luận.

2.6.1 Điều phối bằng đồ thị trạng thái

Thay vì sử dụng các vòng lặp truyền thống với cấu trúc điều kiện cứng nhắc, luồng suy luận của tác tử được mô hình hóa dưới dạng một đồ thị trạng thái có hướng (Directed Stateful Graph) sử dụng thư viện `LangGraph`. Đây là một trong những phương pháp thiết kế tác tử hiện đại nhất trong lĩnh vực LLM Engineering, được sử dụng phổ biến trong các hệ thống sản xuất. Đồ thị gồm hai thành phần cốt lõi:

GraphState - Trạng thái chia sẻ toàn cục:

`GraphState` là một cấu trúc dữ liệu kiểu `TypedDict` chứa toàn bộ thông tin cần thiết trong suốt vòng đời của một lượt xử lý (từ lúc nhận câu hỏi đến lúc trả về kết quả). Mỗi node trong đồ thị đều đọc dữ liệu từ `GraphState` để ra quyết định, và ghi kết quả xử lý trở lại

GraphState để node tiếp theo kế thừa. Cụ thể, trạng thái được phân thành các nhóm trường sau:

- **Nhóm đầu vào:**

- `question`: câu hỏi gốc
- `session_id`: định danh phiên hội thoại

- **Nhóm phân loại ý định** (kết quả do node Intent Gate ghi vào):

- `intent_name`
- `entities`
- `needs_clarification`
- `clarification_question`
- `missing_fields`

- **Nhóm điều khiển vòng lặp:**

- `current_query`: truy vấn hiện tại, có thể thay đổi sau mỗi lần Rewrite
- `all_results`: danh sách tài liệu đã thu thập
- `hop_count`: số vòng tìm kiếm đã thực hiện
- `max_hops`: giới hạn tối đa

- **Nhóm đánh giá:**

- `is_relevant`: tài liệu có liên quan không
- `avg_sim`: điểm tương đồng trung bình
- `eval_reason`: lý do đánh giá — được truyền sang RewriteTool nếu cần viết lại

- **Nhóm kết quả:**

- `raw_answer`: câu trả lời thô từ LLM
- `confidence`: điểm tin cậy
- `gate_action`: quyết định của Confidence Gate
- `final_answer`: câu trả lời cuối cùng sau kiểm duyệt

- **Nhóm theo dõi:**

- `steps`: danh sách các bước suy luận đã thực hiện, phục vụ hiển thị trên giao diện
- `sources`: danh sách nguồn trích dẫn

Các Node — Định xử lý

Đồ thị gồm 7 node, mỗi node là một hàm Python nhận `GraphState` làm đầu vào và trả về một dictionary chứa các trường cần cập nhật trong trạng thái. Bảng dưới đây mô tả chức năng của từng node:

Node	Chức năng	Đầu ra chính ghi vào State
<code>intent_gate</code>	Phân loại ý định, trích xuất thực thể, kiểm tra thông tin thiếu	<code>intent_name</code> , <code>entities</code> , <code>needs_clarification</code>
<code>retrieve</code>	Gọi <code>RetrieveTool</code> tìm kiếm ngữ nghĩa trên ChromaDB	<code>all_results</code> (danh sách tài liệu + điểm)
<code>evaluate</code>	Gọi <code>EvaluateTool</code> đánh giá mức độ liên quan của tài liệu	<code>is_relevant</code> , <code>eval_reason</code> , <code>hop_count</code>
<code>rewrite</code>	Gọi <code>RewriteTool</code> viết lại truy vấn bằng thuật ngữ chuẩn xác hơn; đồng thời reset <code>all_results</code> để xóa tài liệu không liên quan từ vòng trước	<code>current_query</code> (truy vấn mới), <code>all_results</code> (reset)
<code>generate</code>	Gọi <code>GenerateTool</code> tổng hợp câu trả lời từ tập tài liệu	<code>raw_answer</code> , <code>sources</code>
<code>confidence_gate</code>	Tính điểm tin cậy và phân loại Reject/Warn/Pass	<code>confidence</code> , <code>gate_action</code> , <code>final_answer</code>
<code>save_memory</code>	Lưu lượt hội thoại vào Sliding Window Memory	<i>(ghi vào Memory, không cập nhật State)</i>

Conditional Edges — Cạnh điều kiện

Điểm mạnh cốt lõi của LangGraph nằm ở khả năng **rẽ nhánh động** dựa trên trạng thái hiện tại, thay vì đi theo một đường cố định. Đồ thị sử dụng hai hàm định tuyến (routing functions) tại hai điểm phân nhánh then chốt:

Phân nhánh 1 — Sau node `intent_gate` (`route_after_intent`):

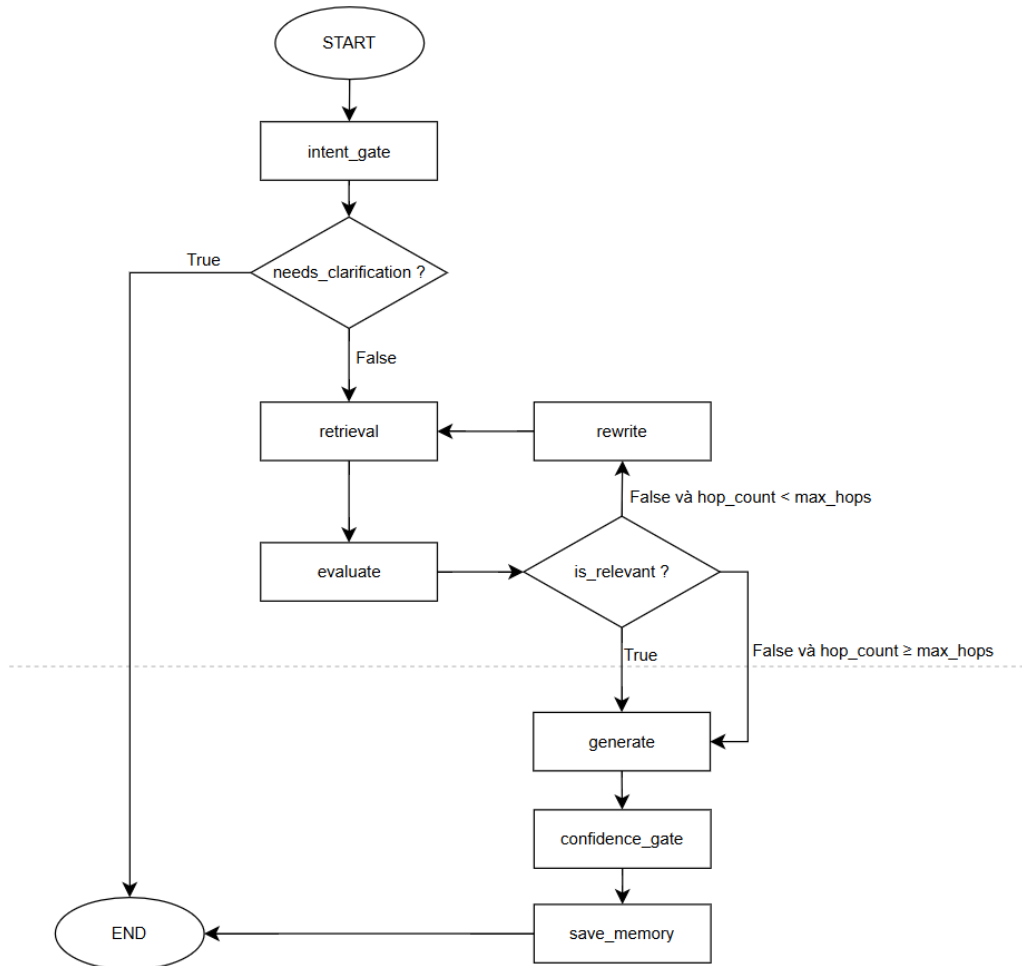
- Nếu `needs_clarification = True` (thiếu thực thể bắt buộc) → kết thúc đồ thị ngay lập tức (END), trả câu hỏi làm rõ về giao diện mà không tiêu tốn tài nguyên tìm kiếm.
- Nếu `needs_clarification = False` (đủ thông tin) → chuyển sang node `retrieve` để bắt đầu vòng lặp RAG.

Phân nhánh 2 — Sau node evaluate (route_after_evaluate):

- Nếu `is_relevant = True` → chuyển sang `generate` vì tài liệu đã đủ chất lượng.
- Nếu `is_relevant = False` và `hop_count >= max_hops` → chuyển sang `generate` (bắt buộc kết thúc dù tài liệu chưa tối ưu, nhường quyền từ chối cho Confidence Gate).
- Nếu `is_relevant = False` và `hop_count < max_hops` → chuyển sang `rewrite`, tạo truy vấn mới rồi quay lại `retrieve`, hình thành **vòng lặp tự sửa lỗi**.

Các cạnh còn lại là cạnh cố định (deterministic edges): `retrieve` → `evaluate`, `rewrite` → `retrieve`, `generate` → `confidence_gate`, `confidence_gate` → `save_memory`, `save_memory` → `END`.

Sơ đồ dưới đây minh họa toàn bộ cấu trúc đồ thị, bao gồm các node, cạnh cố định và hai điểm phân nhánh điều kiện:



Hình 2.11: Đồ thị trạng thái

2.6.2 Thiết kế các công cụ hỗ trợ

Để đảm bảo tính mô-đun hóa và khả năng thay thế độc lập, toàn bộ logic nghiệp vụ trong đồ thị được đóng gói thành các công cụ tuân theo một giao diện chuẩn. Mỗi công cụ nhận đầu

vào đặc thù và trả về một giao diện trừu tượng chuẩn hóa chứa trạng thái thực thi, thông điệp mô tả và dữ liệu kết quả.

Giao diện chuẩn BaseTool

Mọi công cụ trong hệ thống đều kế thừa từ lớp trừu tượng `BaseTool`, định nghĩa ba thành phần bắt buộc:

- **name:** Tên định danh duy nhất của công cụ, được sử dụng để đăng ký và tra cứu trong bộ điều phối.
- **description:** Mô tả ngắn gọn chức năng, phục vụ cho việc ghi nhận trong log hệ thống.
- **execute(**kwargs) → ToolResult:** Hàm thực thi chính, nhận tham số đầu vào và trả về đối tượng `ToolResult` - một cấu trúc dữ liệu chuẩn hóa gồm bốn trường: **success**, **data**, **message** và **metadata**. Việc chuẩn hóa đầu ra giúp các node trong đồ thị LangGraph xử lý kết quả từ mọi công cụ theo cùng một giao thức mà không cần logic phân nhánh riêng cho từng loại.

Hệ thống triển khai bốn công cụ chuyên biệt, mỗi công cụ đảm nhận đúng một tác vụ trong vòng lặp suy luận:

RetrieveTool - Tìm kiếm tài liệu

`RetrieveTool` thực hiện tìm kiếm ngữ nghĩa (semantic search) trên ChromaDB bằng phép đo Cosine Similarity giữa vector truy vấn và các vector tài liệu đã lưu trữ với hai tham số chính là `top_k = 3` và `similarity_threshold = 0.35`.

Công cụ tích hợp cơ chế **Fallback Threshold** (hạ ngưỡng dự phòng): khi số lượng tài liệu trả về dưới ngưỡng tối thiểu, hệ thống tự động hạ thấp ngưỡng tương đồng `similarity_threshold = 0.25` và mở rộng `top_k = 4` để tăng khả năng truy xuất.

EvaluateTool — Đánh giá mức độ liên quan

`EvaluateTool` áp dụng chiến lược đánh giá hai tầng (2-tier evaluation) nhằm cân bằng giữa tốc độ và độ chính xác:

- **Tầng 1 - Kiểm tra thống kê:** Tính điểm tương đồng trung bình (`avg_sim`) của `top_k` tài liệu. Nếu `avg_sim >= min_avg_sim`, hệ thống sẽ gán là nhận diện được tài liệu liên quan.
- **Tầng 2 - Sử dụng LLM:** Nếu như tầng 1 thất bại nhưng vẫn có thể chứa thông tin hữu ích thì sẽ gọi LLM để đánh giá và trả về kết quả JSON, kết quả này sẽ được chuyển cho `RewriteTool` để viết lại câu truy vấn

```

_EVALUATE_PROMPT = """\
Bạn là trợ lý AI về quy chế đào tạo ĐHBK Hà Nội.

Câu hỏi của sinh viên: "{question}"

Dưới đây là các đoạn tài liệu tìm được:
{context}

Nhiệm vụ: Đánh giá xem tài liệu trên có ĐỀ CẬP đến chủ đề câu hỏi hay không.
Lưu ý:
- Tài liệu KHÔNG cần trả lời hoàn chỉnh 100%.
- Chỉ cần chứa thông tin liên quan là ĐỦ (relevant=true).
- Chỉ relevant=false nếu hoàn toàn khác chủ đề.

Trả về JSON duy nhất:
{"relevant": true/false, "reason": "Lý do ngắn gọn"}"""

```

Hình 2.12: Câu truy vấn dùng LLM đánh giá

Chiến lược hai tầng này đảm bảo rằng LLM chỉ được gọi khi thực sự cần thiết, giảm thiểu độ trễ cho các truy vấn đơn giản trong khi vẫn duy trì khả năng đánh giá sâu cho các trường hợp phức tạp.

RewriteTool - Viết lại truy vấn

`RewriteTool` chỉ được gọi khi `EvaluateTool` xác định tài liệu chưa đủ liên quan và `hop_count < max_hops`. Công cụ hai tham số là `question` và `reason` từ `EvaluateTool` đã nói ở trên để viết lại câu.

```

_REWRITE_PROMPT = """\
Bạn là chuyên gia về quy chế đào tạo ĐHBK Hà Nội.

Câu hỏi gốc: "{question}"
Lý do tìm kiếm chưa đủ: "{reason}"

Hãy viết lại câu hỏi dùng thuật ngữ pháp lý/học thuật chính xác hơn.
Chỉ trả về 1 câu duy nhất, KHÔNG giải thích.

Ví dụ:
- "trượt 14 tín" → "cảnh cáo học tập tín chỉ nợ không đạt"
- "bị đuổi học" → "buộc thôi học do điểm tích lũy thấp"

Câu viết lại:"""

```

Hình 2.13: Câu truy vấn dùng LLM viết lại câu

GenerateTool — Tổng hợp câu trả lời

`GenerateTool` là công cụ cuối cùng trong chuỗi suy luận, chịu trách nhiệm tổng hợp câu trả lời hoàn chỉnh từ tập tài liệu đã thu thập. Công cụ xây dựng ngữ cảnh bằng cách ghép nối nội dung các tài liệu, kèm theo siêu dữ liệu chi tiết cho mỗi đoạn: tên nguồn, tiêu đề chương, tiêu đề điều và điểm tương đồng.


```

_ANSWER_PROMPT = """\
{system_prompt}

Dựa trên các đoạn tài liệu quy chế dưới đây, hãy trả lời câu hỏi.
Hãy phát hiện ngôn ngữ của câu hỏi và trả lời bằng ĐÚNG ngôn ngữ đó.

=== TÀI LIỆU THAM KHẢO ===
{context}

=== CÂU HỎI ===
{question}

=== YÊU CẦU ===
- Trả lời trực tiếp, rõ ràng, đúng trọng tâm
- Trích dẫn cụ thể số Điều, Chương, tên văn bản nếu có
- KHÔNG bịa đặt thông tin ngoài tài liệu
- Nếu thông tin không đủ, thừa nhận giới hạn

=== CÂU TRẢ LỜI ==="""

```

Hình 2.14: LLM tổng hợp và tạo câu trả lời hoàn chỉnh

Kiến trúc này cho phép Agent hoạt động như một chu trình **Tìm kiếm – Đánh giá – Tự sửa lỗi**, phản ánh đúng bản chất của một hệ thống Agentic RAG đa bước - vượt xa khả năng của các hệ thống RAG truyền thống tuyến tính.

2.7 Thiết kế tầng bộ nhớ

Trong bất kỳ hệ thống hỏi-đáp thông minh nào, khái niệm "bộ nhớ" cần được hiểu theo hai chiều khác nhau: tri thức tĩnh về lĩnh vực chuyên môn và tri thức động về ngữ cảnh hội thoại đang diễn ra. Tầng bộ nhớ đóng vai trò như một lớp hạ tầng lưu trữ phục vụ đồng thời cho cả tầng xử lý dữ liệu lẫn tầng suy luận tác tử, mà không trực tiếp tham gia vào luồng xử lý chính.

2.7.1 Cơ sở dữ liệu vector — Tri thức tĩnh

Thành phần đầu tiên của tầng bộ nhớ là cơ sở dữ liệu vector được xây dựng hoàn toàn trong pha ngoại tuyến và không thay đổi trong suốt pha trực tuyến. Như đã trình bày tại mục 2.4, toàn bộ tài liệu quy chế sau khi được xử lý sẽ được lưu trữ bền vững vào ChromaDB tại thư mục `./data/chroma/`.

Trong pha trực tuyến, ChromaDB sẽ cung cấp tri thức chuyên môn theo yêu cầu của `RetrieveTool`. Do bản chất hoàn toàn tĩnh - không có thao tác ghi nào trong quá trình vận hành - ChromaDB đảm bảo tính nhất quán và an toàn của cơ sở tri thức trong mọi phiên làm việc đồng thời.

2.7.2 Bộ nhớ hội thoại — Tri thức động

Thành phần thứ hai, quan trọng hơn về mặt thiết kế hệ thống, là cơ chế bộ nhớ cửa sổ trượt (Sliding Window Memory) phục vụ quản lý ngữ cảnh hội thoại động.

ConversationTurn - Đơn vị lưu trữ

Mỗi lượt hội thoại được đóng gói thành một đối tượng `ConversationTurn` với các trường sau:

```
@dataclass
class ConversationTurn:
    question: str          # Câu hỏi của người dùng
    answer: str            # Câu trả lời của hệ thống (≤ 300 ký tự)
    entities: Dict[str, Any] # Thực thể đã bóc tách (VD: {nganh_hoc: "CNTT"})
    intent_name: str        # Tên intent đã phân loại (VD: "LANGUAGE_REQUIREMENT")
    needs_clarification: bool # True nếu lượt này là câu hỏi làm rõ
```

Hình 2.15: Lớp hội thoại

Việc lưu kèm `entities` và `intent_name` cho phép hệ thống không chỉ nhớ nội dung câu hỏi mà còn nhớ thông tin có cấu trúc đã được trích xuất.

Sliding Window - Cửa sổ trượt

`ConversationMemory` lưu trữ lịch sử hội thoại trong một cấu trúc Python dictionary, ánh xạ `session_id` → `List[ConversationTurn]`. Khi thêm một lượt mới, nếu số lượt vượt quá ngưỡng `window_size`, danh sách sẽ được cắt để chỉ giữ lại K lượt gần nhất. Hệ thống sẽ tránh được vấn đề bộ nhớ tăng trưởng vô hạn, tránh tăng chi phí và giữ được độ chính xác của mô hình

API - Giao diện công khai

Module `ConversationMemory` cung cấp sáu phương thức được phân chia rõ ràng thành hai nhóm đọc và ghi, phục vụ cho các thành phần khác nhau trong hệ thống:

- `add_turn(session_id, ...)` - Ghi một lượt hội thoại mới vào bộ nhớ; được gọi bởi cổng hậu xử lý sau khi câu trả lời được kiểm duyệt.
- `get_context(session_id)` - Tổng hợp K lượt gần nhất thành một đoạn văn bản ngắn gọn để đưa vào câu truy vấn của cổng tiền xử lý và `RewriteTool`, giúp LLM hiểu ngữ cảnh cuộc trò chuyện.
- `get_entities_from_memory(session_id)` - Gộp và trả về toàn bộ thực thể đã được xác định trong cửa sổ K lượt, cho phép tự điền các thực thể mà đã lưu từ hội thoại trước mà không cần hỏi lại người dùng.
- `get_last_clarification_intent(session_id)` - Truy xuất ý định của lượt hội thoại gần nhất có cờ `needs_clarification = True`; được sử dụng để xử lý đúng trường hợp người dùng trả lời câu hỏi làm rõ (ví dụ: sau khi hệ thống hỏi "*Bạn học Khóa nào?*", người dùng trả lời "*Mình K68*" - lúc này hệ thống cần biết ý định gốc là gì để tiếp tục truy vấn).

Nhờ `get_entities_from_memory()`, cổng tiền xử lý ở lượt 2 có thể ghép hai mảnh thông tin này lại và chuyển thẳng sang tác tử mà không cần người dùng gõ lại câu hỏi hoàn chỉnh.

Thiết kế này mang lại trải nghiệm hội thoại tự nhiên và liên tục, đồng thời giữ chi phí lưu trữ ở mức tối thiểu bằng cách chỉ duy trì một cửa sổ ngữ cảnh có giới hạn thay vì toàn bộ lịch sử hội thoại.

Ví dụ minh họa:

Người dùng ở lượt 1 hỏi: "*Chuẩn ngoại ngữ đầu ra là bao nhiêu?*"

Hệ thống hỏi lại: "*Bạn đang học Khóa nào?*".

Lượt 2 người dùng trả lời: "*Mình K68*".

Chương 3

Thực nghiệm và đánh giá

3.1 Môi trường và cấu hình triển khai

3.1.1 Cấu hình phần cứng và phần mềm

Hệ thống được phát triển và thực nghiệm trong môi trường cục bộ trên máy tính cá nhân, không yêu cầu máy chủ đám mây hay GPU chuyên dụng, nhằm ưu tiên tính đơn giản trong triển khai và bảo mật dữ liệu quy chế trong quá trình xây dựng.

Cấu hình phần cứng thực nghiệm

Thành phần	Cấu hình
CPU	Intel Core i5-11300H
RAM	16 GB DDR4
Ổ cứng	SSD 512 GB
GPU	Không sử dụng (mô hình embedding chạy trên CPU)
Hệ điều hành	Windows 11

Bảng 3.1: Cấu hình phần cứng

Các tham số cấu hình hệ thống chính

```
retrieval:
  top_k: 3 # Số tài liệu trả về mỗi lần truy xuất
  similarity_threshold: 0.35 # Ngưỡng cosine similarity tối thiểu

agent:
  max_iterations: 5 # Số vòng lặp tối đa của đồ thị Agent
  high_confidence_threshold: 0.65 # Ngưỡng PASS của Confidence Gate ( $\geq 65\%$ )
  low_confidence_threshold: 0.35 # Ngưỡng REJECT của Confidence Gate ( $< 35\%$ )
  min_avg_similarity: 0.45 # Ngưỡng avg_sim để EvaluateTool kết luận đủ tài liệu

memory:
  window_size: 5 # Số lượt Q&A tối đa trong Sliding Window
  max_context_chars: 1500 # Giới hạn ký tự ngữ cảnh inject vào prompt LLM
```

Hình 3.1: Cấu hình hệ thống

3.1.2 Cấu hình mô hình

Mô hình nhúng

Sử dụng mô hình BAAI/bge-m3 tải từ trang HuggingFace, chạy cục bộ trên CPU. Đây là lựa chọn phù hợp vì mô hình hỗ trợ đa ngôn ngữ mạnh, cho phép toàn bộ Pha ngoại tuyến (xây dựng vector database) không phụ thuộc vào bất kỳ API bên ngoài nào.

Đặc điểm	Giá trị
Số chiều vector	1024 chiều
Phương pháp đo tương đồng	Cosine Similarity
Batch size khi build DB	32
Môi trường thực thi	CPU, không cần GPU
Phạm vi dùng	Pha ngoại tuyến — không gọi trong pha trực tuyến

Mô hình ngôn ngữ lớn

Hệ thống được thiết kế để hỗ trợ linh hoạt hai chế độ LLM thông qua một tham số `provider` duy nhất trong `config.yaml`, có thể chuyển đổi mà không cần sửa mã nguồn:

```

llm:
  provider: "gemini"           # Đổi thành "ollama" để dùng model cục bộ
  model_name: "gemini-2.5-flash"
  temperature: 0.3           # Sáng tạo thấp để đảm bảo tính nhất quán
  max_tokens: 2048
  timeout_seconds: 120
  api_key_env: "GEMINI_API_KEY"

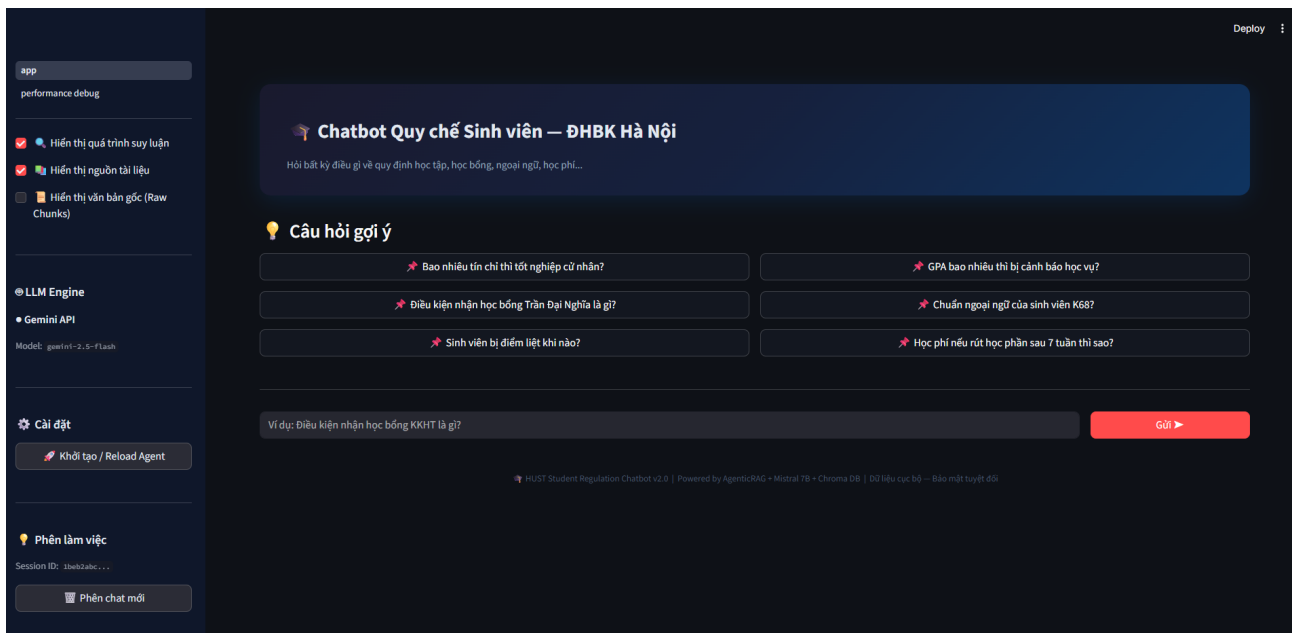
```

Hình 3.2: Tham số mô hình ngôn ngữ lớn

Chế độ	Provider	Model	Đặc điểm
Cục bộ (Local)	Ollama	Tùy chọn (VD: Mistral 7B, Gemma)	Bảo mật tuyệt đối, không phụ thuộc Internet, không tốn phí API, nhưng yêu cầu phần cứng đủ mạnh (GPU $\geq$ 8GB VRAM)
Đám mây (Cloud)	Gemini API	gemini-2.5-flash	Chất lượng suy luận cao, tốc độ ổn định, nhưng dữ liệu phải gửi ra ngoài qua Internet

Bảng 3.2: Tùy chọn sử dụng theo hai chế độ

Về mặt nguyên tắc thiết kế, phương án lý tưởng cho một hệ thống chatbot xử lý quy chế nội bộ là triển khai LLM trên server của trường để đảm bảo tính bảo mật. Nhưng do đề án bị giới hạn về phần cứng nên thay vì chạy mô hình cục bộ thì sẽ lựa chọn thay thế là dùng Gemini API để phục vụ cho việc kiểm thử và đánh giá.



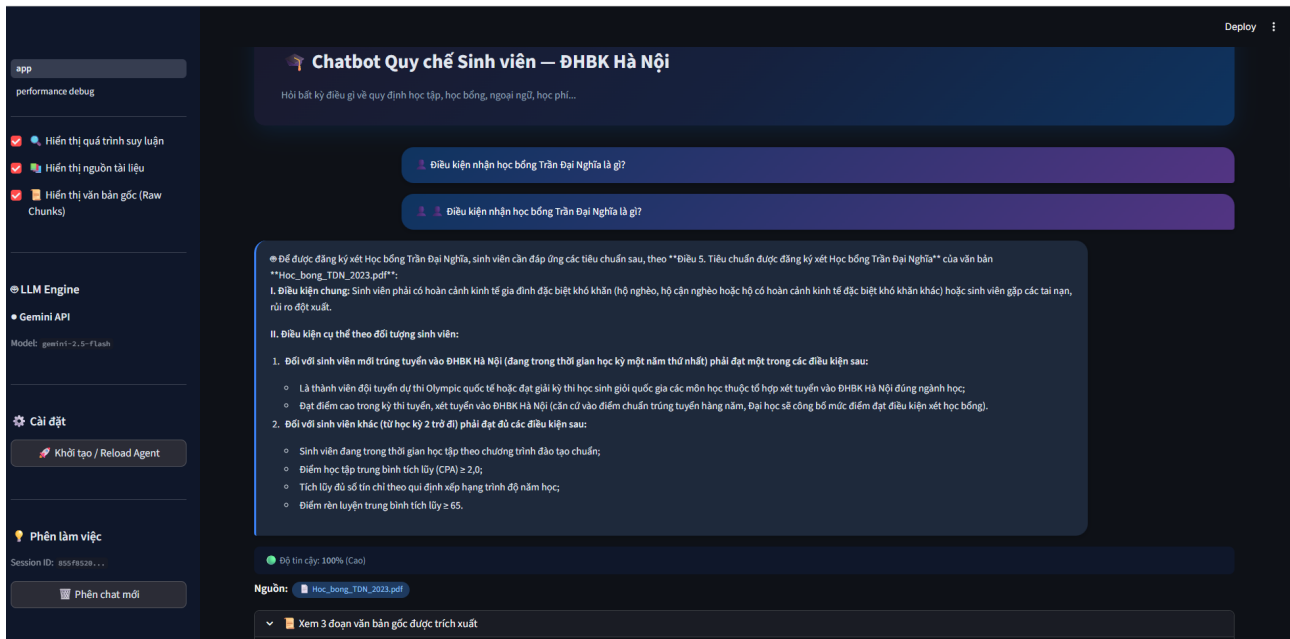
Hình 3.3: Giao diện hệ thống

3.2 Các kịch bản thử nghiệm

3.2.1 Truy vấn đơn giản

Đối với các câu hỏi đơn giản, áp dụng với toàn trường thì hệ thống sẽ không cần hỏi lại thông tin từ người dùng mà sẽ trả lời luôn.

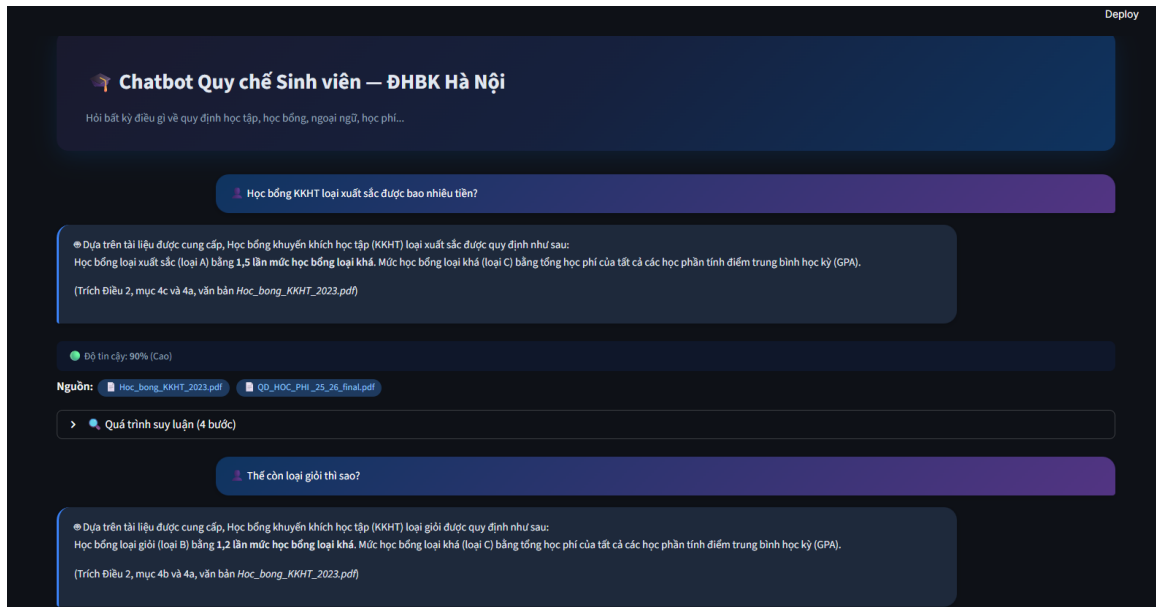
UC1: Hỏi về chính sách học bổng Trần Đại Nghĩa



Hình 3.4: UC1: Hỏi về chính sách học bổng

3.2.2 Hội thoại đa lượt

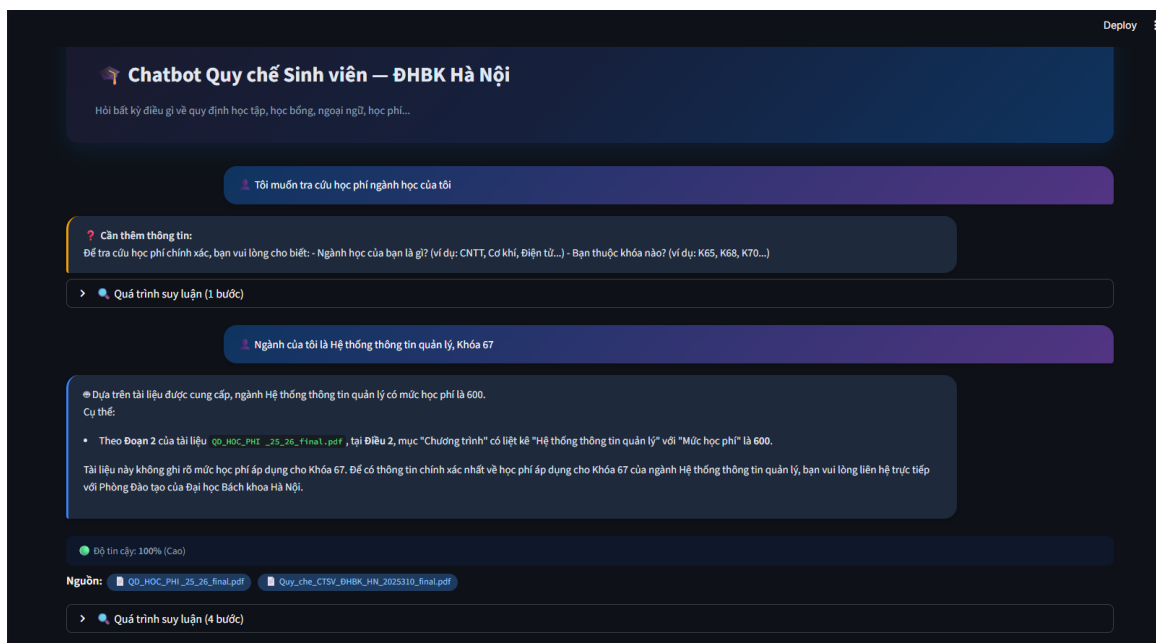
UC2: Hỏi đáp nối tiếp dựa trên ngữ cảnh



Hình 3.5: Hỏi đáp nối tiếp dựa trên ngữ cảnh

UC2.1: Bổ sung thực thể

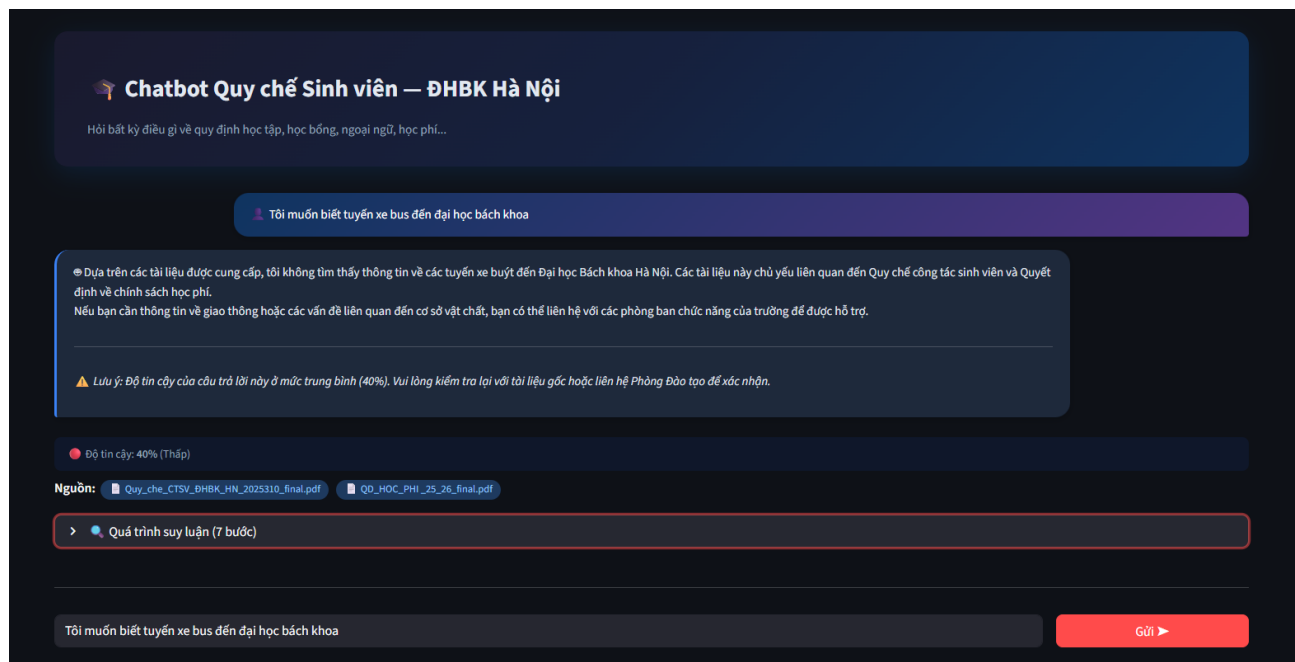
Đối với những câu hỏi thuộc các nhóm ý định yêu cầu người dùng bổ sung thêm các thông tin khác thì hệ thống sẽ gửi tin nhắn yêu cầu bổ sung:



Hình 3.6: Hỏi về học phí sẽ cần thông tin về ngành học

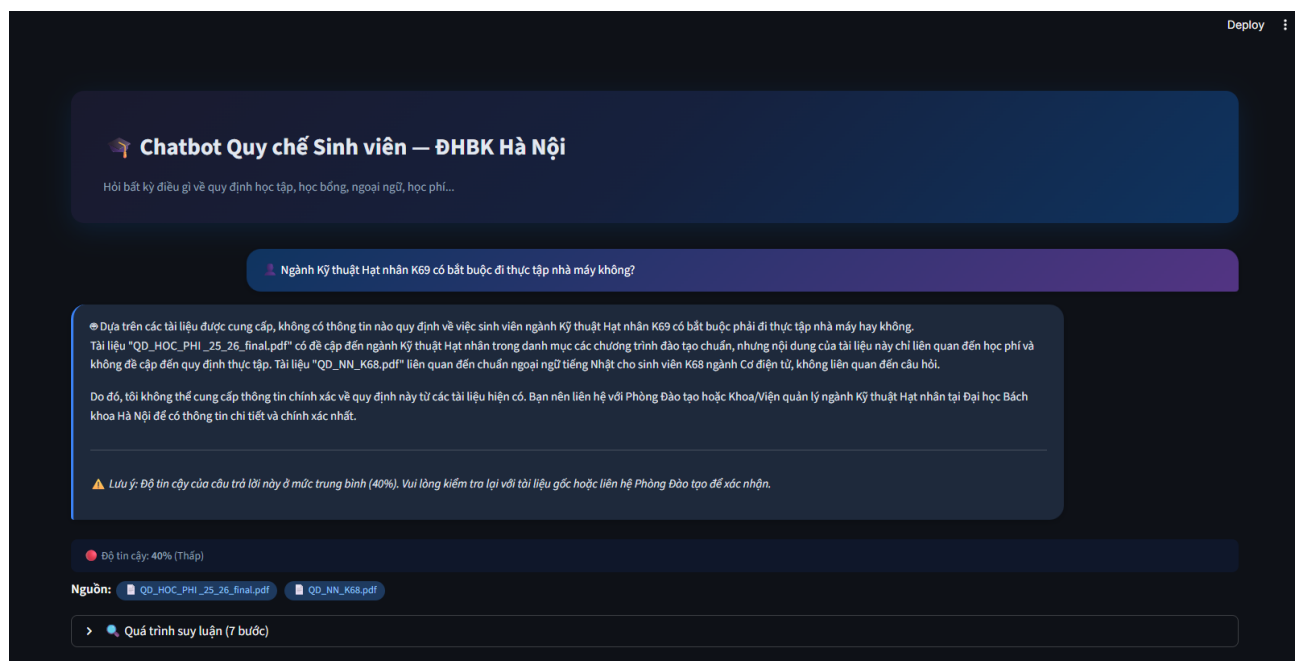
3.2.3 Câu hỏi ngoài phạm vi

UC3: Hỏi những vấn đề không có trong cơ sở tri thức



Hình 3.7: Hệ thống từ chối trả lời do không có thông tin

UC3.1: Câu hỏi về ngành học cụ thể nhưng chưa có file quy định riêng



Hình 3.8: Hệ thống từ chối trả lời do không có thông tin

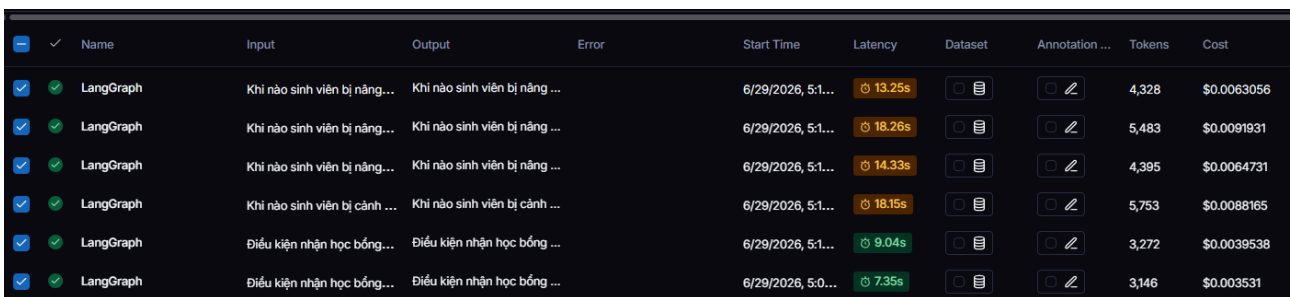
3.3 Phương pháp đánh giá

3.3.1 Đánh giá định lượng

Sử dụng LangSmith đã được cấu hình từ trước để đánh giá kết quả định lượng của hệ thống dựa trên từng nhóm kịch bản đã trình bày. Đánh giá sẽ dựa theo hai tiêu chí là thời gian phản hồi và lượng token đã sử dụng, và như đã trình bày ở trên hệ thống sẽ được đánh giá trên mô hình `gemini-2.5-flash`

- **UC1 - Truy vấn đơn giản:** Nhóm những câu hỏi không phức tạp, chỉ cần một lần truy xuất là có thể trả ra kết quả

1. Lượng token: 3.000 - 5000 token
2. Thời gian phản hồi: 10 - 15s

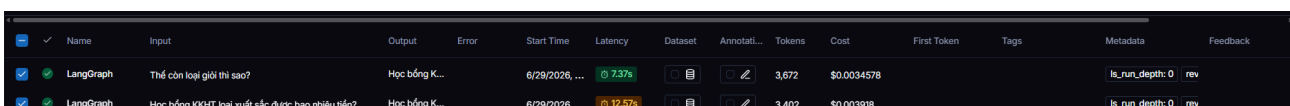


	Name	Input	Output	Error	Start Time	Latency	Dataset	Annotation ...	Tokens	Cost
✓	LangGraph	Khi nào sinh viên bị nâng...	Khi nào sinh viên bị nâng ...		6/29/2026, 5:1...	13.25s			4,328	\$0.0063056
✓	LangGraph	Khi nào sinh viên bị nâng...	Khi nào sinh viên bị nâng ...		6/29/2026, 5:1...	18.26s			5,483	\$0.0091931
✓	LangGraph	Khi nào sinh viên bị nâng...	Khi nào sinh viên bị nâng ...		6/29/2026, 5:1...	14.33s			4,395	\$0.0064731
✓	LangGraph	Khi nào sinh viên bị cảnh ...	Khi nào sinh viên bị cảnh ...		6/29/2026, 5:1...	18.15s			5,753	\$0.0088165
✓	LangGraph	Điều kiện nhận học bổng...	Điều kiện nhận học bổng ...		6/29/2026, 5:1...	9.04s			3,272	\$0.0039538
✓	LangGraph	Điều kiện nhận học bổng...	Điều kiện nhận học bổng ...		6/29/2026, 5:0...	7.35s			3,146	\$0.003531

Hình 3.9: Kết quả phân tích

- **UC2 - Hội thoại đa lượt:** Bản chất của câu hỏi sẽ giống như hỏi hai câu truy vấn đơn giản, nên tài nguyên sử dụng để trả lời từng câu hỏi không quá chênh lệch.

1. Lượng token: 3.500 - 4.000 token mỗi câu
2. Thời gian phản hồi: 10 - 15s mỗi câu

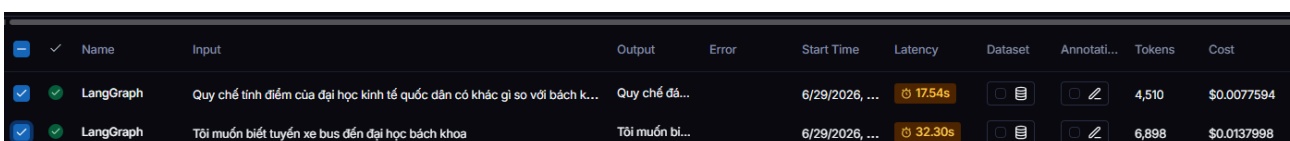


	Name	Input	Output	Error	Start Time	Latency	Dataset	Annotati...	Tokens	Cost	First Token	Tags	Metadata	Feedback
✓	LangGraph	Thế còn loại giò thì sao?	Học bổng K...		6/29/2026, ...	7.37s			3,672	\$0.0034578			is_run_depth: 0	rev
✓	LangGraph	Học bổng KCHT loại xuất sắc được bao nhiêu tiền?	Học bổng K...		6/29/2026, ...	12.57s			3,402	\$0.003918			is_run_depth: 0	rev

Hình 3.10: Kết quả phân tích

- **UC3 - Câu hỏi ngoài phạm vi:** Các câu hỏi ngoài phạm vi sẽ tốn nhiều tài nguyên và thời gian hơn do phải trải qua 2 vòng lặp đánh giá như đã trình bày ở mục 2.6.1

1. Lượng token: 4.500 - 7.000 token
2. Thời gian phản hồi: 20 - 30s



	Name	Input	Output	Error	Start Time	Latency	Dataset	Annotati...	Tokens	Cost
✓	LangGraph	Quy chế tính điểm của đại học kinh tế quốc dân có khác gì so với bách k...	Quy chế đả...		6/29/2026, ...	17.54s			4,510	\$0.0077594
✓	LangGraph	Tôi muốn biết tuyến xe bus đến đại học bách khoa	Tôi muốn bi...		6/29/2026, ...	32.30s			6,898	\$0.0137998

Hình 3.11: Kết quả phân tích

3.3.2 Đánh giá định tính

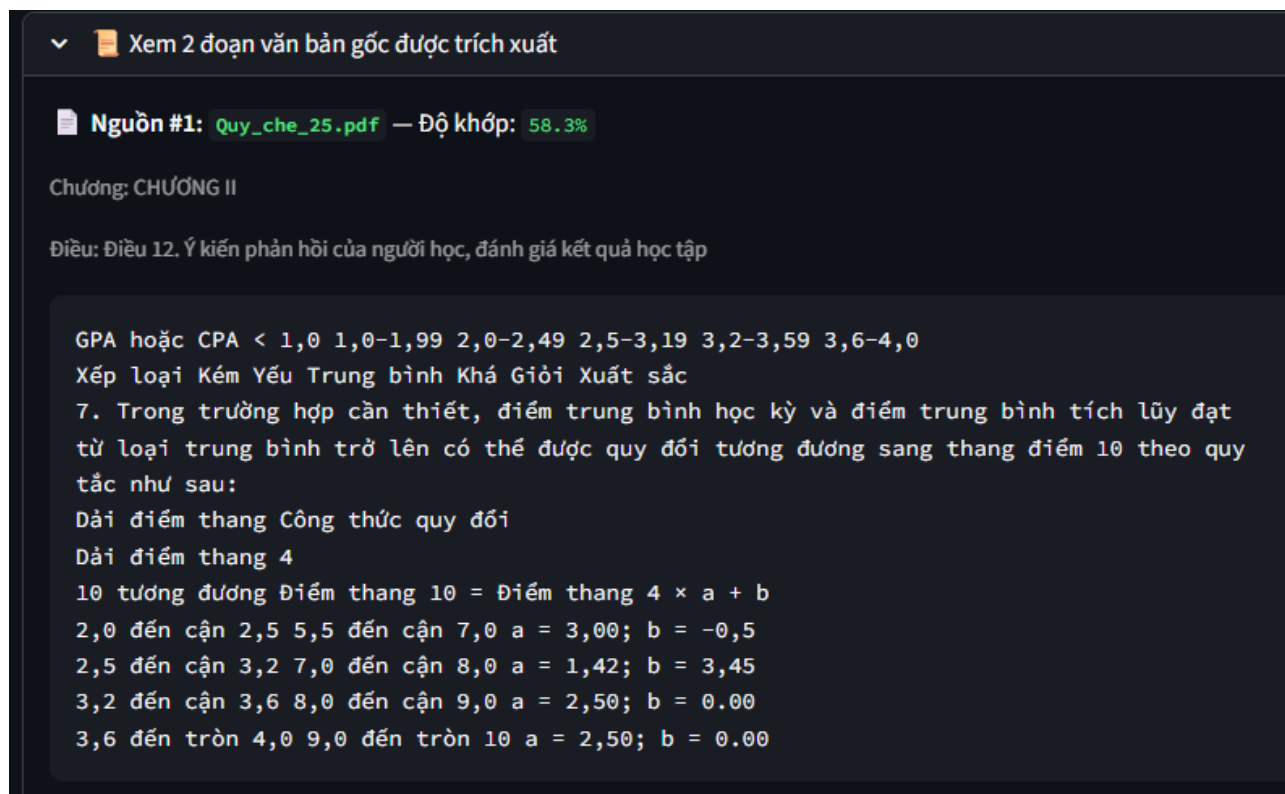
Bên cạnh các kết quả định lượng như đã trình bày trên, đánh giá định tính giúp kiểm tra xem hệ thống có trả lời đúng chính xác so với quy chế gốc và khả năng trích nguồn tham khảo.

- **Tính chính xác của câu trả lời:** Sau khi thử nghiệm trên các kịch bản thì hệ thống trả lời tốt đối với những câu hỏi đơn giản, biết từ chối trả lời câu hỏi không biết và có khả năng hỏi đáp đa lượt.



Hình 3.12: Hệ thống từ chối trả lời

- **Khả năng truy vết:** Các câu hỏi đều được trích ngữ cảnh mà hệ thống đã tìm được, giúp người dùng có thể tự kiểm tra.



Hình 3.13: Khả năng truy vết

Kết luận

Kết quả đạt được

Qua quá trình nghiên cứu và thực hiện, đề án “**Xây dựng chatbot hỏi đáp quy chế sinh viên**” đã hoàn thành các mục tiêu đề ra ban đầu, giải quyết thành công bài toán tra cứu thông tin học vụ đặc thù:

- **Xây dựng quy trình ETL:** Hoàn thiện quy trình xử lý dữ liệu đầu vào từ các văn bản PDF quy chế chính thức của Nhà trường. Quy trình bao gồm làm sạch, chuẩn hóa và phân đoạn văn bản thông minh, đảm bảo giữ nguyên tính toàn vẹn của các điều khoản, quy định phức tạp trong môi trường đào tạo tín chỉ.
- **Tối ưu hóa hệ thống truy xuất tri thức (Retrieval):** Thiết kế và triển khai cơ sở dữ liệu vector với ChromaDB, tích hợp mô hình embedding ngữ nghĩa được tinh chỉnh cho tiếng Việt chuyên ngành. Điều này giúp hệ thống hiểu và truy xuất chính xác các văn bản hành chính có cấu trúc phức tạp của Bách Khoa.
- **Thiết kế và triển khai kiến trúc Agentic RAG với đồ thị trạng thái LangGraph:** Nâng cấp từ RAG tuyến tính sang kiến trúc Agent suy luận đa bước, được điều phối bởi StateGraph gồm 6 node (Intent Gate → Retrieve → Evaluate → Rewrite → Generate → Confidence Gate). Hệ thống có khả năng tự đánh giá chất lượng tài liệu truy xuất, tự viết lại truy vấn bằng thuật ngữ pháp lý chuẩn để tối ưu câu trả lời.
- **Xây dựng cổng tiền xử lý - Intent Gate:** Thiết kế bộ phân loại ý định hỗ trợ 5 nhóm câu hỏi chuyên biệt. Hệ thống tự động phát hiện các thực thể còn thiếu và chủ động đặt câu hỏi làm rõ thay vì truy xuất sai mục tiêu, đảm bảo câu trả lời luôn gắn đúng với đối tượng áp dụng cụ thể.
- **Xây dựng cổng hậu xử lý - Confidence Gate:** Xây dựng thuật toán tính điểm tin cậy tổng hợp ba yếu tố độc lập: số lượng tài liệu tương đối, mức độ cụ thể của câu trả lời, số vòng lặp suy luận. Nhằm phân loại đầu ra thành ba trạng thái: Pass, Warn và Reject. Giúp hệ thống có thể kiểm soát đầu ra.

Hướng phát triển của đề án

Để hệ thống thực sự trở thành trợ lý ảo đắc lực cho sinh viên Bách Khoa, đề án đề xuất các hướng phát triển nâng cao trong tương lai:

- **Nâng cấp EvaluateTool để xử lý câu hỏi đa khía cạnh:** Bổ sung cơ chế đánh giá mức độ bao phủ thay vì chỉ đánh giá mức độ liên quan đơn thuần. Khi một câu hỏi có nhiều điều kiện ràng buộc, tác tử sẽ nhận biết được các khía cạnh còn thiếu tài liệu và tiếp tục vòng lặp tìm kiếm thay vì chuyển sang sinh câu trả lời khi mới chỉ đáp ứng một phần câu hỏi.
- **Mở rộng và cập nhật cơ sở tri thức động:** Phát triển quy trình bổ sung tài liệu tự động khi đại học ban hành văn bản quy chế mới mà không cần xây dựng lại toàn bộ cơ sở dữ liệu. Đồng thời mở rộng phạm vi tài liệu sang các lĩnh vực liên quan như lịch thi, thông báo học vụ định kỳ và hướng dẫn đăng ký học phần.
- **Tích hợp vào hệ thống thông tin đào tạo của đại học:** Đóng gói hệ thống dưới dạng REST API (FastAPI) để tích hợp vào cổng thông tin sinh viên, ứng dụng di động hoặc các kênh giao tiếp hiện có (Zalo OA, Microsoft Teams). Thiết kế theo hướng module hóa cho phép các trường đại học khác dễ dàng kế thừa và tái sử dụng toàn bộ hệ thống cho cơ sở tri thức riêng của mình.
- **Tối ưu hóa hạ tầng và độ trễ:** Nghiên cứu các giải pháp tối ưu hóa suy luận (Inference optimization) và lựa chọn hạ tầng phần cứng phù hợp để đảm bảo tốc độ phản hồi nhanh, chịu tải tốt, đặc biệt trong các đợt cao điểm như đăng ký tín chỉ hay xét tốt nghiệp.

Do thời gian thực hiện và nguồn lực tính toán còn hạn chế, báo cáo khó tránh khỏi những thiếu sót nhất định. Em rất mong nhận được những ý kiến đóng góp quý báu của Thầy/Cô và các bạn để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn!

Tài liệu tham khảo

- [1] LangChain. LangChain – framework for developing applications powered by language models, 2024.
- [2] LangChain AI. LangGraph: Build resilient language agents as graphs, 2024.
- [3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *arXiv preprint arXiv:2005.11401*, 2020.
- [4] Anu Madan. Mcp vs rag: Competitors or complements?, 2025.
- [5] Qwen Team. Qwen2.5-LLM: Extending the boundary of LLMs, 2025.
- [6] Anh Tuan. Agentic RAG – bước tiến tiếp theo của Retrieval-Augmented Generation?, 2025.
- [7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, Long Beach, CA, USA, 2017.